



Análise de Séries Temporais

ENG1469

Lista Prática 02

Aluno(a)	Paloma Fernanda Loureiro Sette - 1621595
Professor(a)	Cristiano Fernandes

Rio de Janeiro, 20 de Outubro de 2024

Sumário

Lista de Figuras	1
0.1 Carregando os dados	4
0.2 Especificações	5
1 - Questão 01	6
1.1 Questão 1: Parte (i)	6
1.1.1 Análise Gráfica da Série Temporal e Diferenças	6
1.1.2 Expressão da Série Diferenciada	8
1.1.3 Justificativa do Valor de $d = 1$	8
1.2 Questão 1: Parte (ii)	8
1.2.1 Teste KPSS para verificar a estacionaridade da série e determinar d	8
1.2.2 AC (Função de Autocorrelação) e FACP (Função de Autocorrelação Parcial)	9
1.2.3 (a) Existe dependência sazonal nessa série?	10
1.2.4 (b) É possível sugerir um modelo ARIMA(p,d,q), de partida, para a série y_t ?	10
1.2.5 (c) O padrão de decaimento da FAC nos lags sazonais sugere a necessidade de diferenciação sazonal “D” para a série, além de “d”?	11
1.3 Questão 1: Parte (iii)	11
1.3.1 (a) Melhor modelo e valor do AICc	12
1.3.2 (b) Análise da FAC e FACP dos resíduos	13
1.4 Questão 1: Parte (iv)	15
1.4.1 (a) Quantos modelos foram comparados para selecionar o modelo sub-ótimo?	18
1.4.2 (b) AICc e a equação SARIMA(p,d,q)(P,D,Q) em termos das ordens p, q, d, D, P e Q	18
1.4.3 (c) FAC e FACP dos resíduos e o resultado do teste Ljung-Box para $m = 20$	19
1.5 Questão 1: Parte (v)	21
1.5.1 (a) Contrastando o número de modelos que foram comparados na busca do modelo ótimo com aquele obtido na busca do modelo sub-ótimo efetuada em (iv)	24
1.5.2 (b) AICc e Expressão dos Modelos	24
1.5.3 (c) FAC/FACP dos resíduos e o resultado do teste Ljung-Box para $m=20$	26
1.6 Questão 1: Parte (vi)	27
1.6.1 (a) A Expressão SARIMA(p,d,q)(P,D,Q) de Cada Modelo	27
1.6.2 (b) O Valor do AICc de Cada Modelo	27
1.6.3 (c) Valores dos coeficientes de autocorrelação dos resíduos e teste Ljung-Box para $m=20$	29
2 Questão 2	32
2.1 Obtendo as Previsões	32
2.1.1 Organizando as Previsões	34
2.2 Calculando as Métricas de Aderência	34
2.2.1 Organizando as Métricas	36
2.3 Analisando os Resultados	37

3	Questão 3	38
3.1	Questão 3: Parte (a)	38
3.2	Questão 3: Parte (b)	39

Lista de Figuras

1	Ipeadata - caminho até a base de dados	2
2	Ipeadata - base de dados utilizada	2
3	Dataframe base - ST	4
4	Gráficos plotados da série e primeira e segunda diferenças	7
5	Gráfico do site Ipeadata	7
6	Resultado - KPSS	9
7	Gráficos da FAC e FACP	10
8	Performing stepwise search to minimize aicc	12
9	Tabela de resultados - SARIMA	13
10	Gráficos da FAC e da FACP dos resíduos	14
11	Tabela de resultados - SARIMA	18
12	Gráficos da FAC e da FACP dos resíduos do modelo SARIMA(3,1,1)(1,0,1) ₁₂	20
13	Tabela de resultados: execução extensa - SARIMA	24
14	Gráficos da FAC e FACP dos resíduos do modelo ótimo obtido	26
15	Saída do Programa: Valor do AICc de cada modelo	28
16	Saída do Programa: coeficientes de autocorrelação e Ljung-Box para o modelo sazonal	30
17	Saída do Programa: coeficientes de autocorrelação e Ljung-Box para o modelo sub-ótimo	30
18	Saída do Programa: coeficientes de autocorrelação e Ljung-Box para o modelo ótimo	31
19	Saída do Programa: previsões para os modelos SARIMA	33
20	Previsões para os modelos ETS do trabalho 01	34
21	Saída do Programa: métricas de aderência para os modelos SARIMA e métodos ingênuos	36
22	Saída do Programa: métricas de aderência para os modelos ETS	36
23	Saída do Programa: Raízes do polinômio de Média Móvel e Invertibilidade	39
24	Saída do Programa: Dataframe com as previsões para toda a série - SARIMA	43
25	Saída do Programa: Dataframe com as previsões para toda a série - ETS	44
26	Gráfico comparativo das previsões - modelos ETS e SARIMA	45

Introdução

Este relatório refere-se à segunda lista prática da disciplina de Análise de Séries Temporais (ENG1469) de 2024.2.

Para este trabalho, utilizaremos a série temporal relacionada a vendas nominais no varejo de livros, jornais, revistas e papelaria. É possível encontrar esta série no [Ipeadata](http://ipeadata.gov.br). (macroeconômico » Periodicidade » Mensal » Consumo e Vendas » Vendas nominais no varejo de livros, jornais, revistas e papelaria). Abaixo, apresentamos uma breve demonstração de como chegar até a base de dados utilizada. Esta demonstração se deve ao fato de que, por algum motivo, o site do Ipeadata não nos gera o link exato até a base (ou seja, qualquer endereço que copiamos é sempre <http://ipeadata.gov.br/Default.aspx>).



Figura 1: Ipeadata - caminho até a base de dados



Figura 2: Ipeadata - base de dados utilizada

Para vendas nominais, considera-se a receita bruta de revenda, total e por Unidade da Federação, definida no âmbito da empresa como a receita bruta mensal proveniente da

revenda de mercadorias, não deduzidos os impostos incidentes e nem as vendas canceladas, abatimentos e impostos incondicionais. Para varejo considera-se as atividades de revenda, venda sem transformação significativa, de bens de consumo novos e usados, preponderantemente para o consumidor final. O comércio varejista é organizado para vender mercadorias em pequenas quantidades ao consumidor final, representando, portanto, o último elo da cadeia de distribuição. A classificação das atividades do comércio varejista baseia-se na gama de produtos vendidos, sem distinção da forma de comercialização em loja ou fora de loja. Para livros, jornais, revistas e papelaria considera-se o comércio varejista de livros, inclusive didáticos, o comércio varejista de revistas, jornais, revistas e outros impressos, o comércio varejista de artigos de escritório, as bancas de jornais e revistas, o comércio varejista de artigos de papelaria, tais como cadernos, borrachas, lápis, canetas, cartolinas e similares. Não fazendo parte desta classificação o comércio varejista de livros usados, o comércio varejista de selos, o comércio varejista de suprimentos de informática. A base foi fixada como a média das observações no ano de 2022. Nota: A Pesquisa Mensal de Comércio foi iniciada em janeiro de 1995, com o intuito de produzir indicadores que possam acompanhar o desempenho conjuntural do comércio varejista no país, investigando a receita bruta de revenda nas empresas formalmente constituídas de 20 ou mais pessoas ocupadas.

0.1 Carregando os dados

A princípio, é necessário carregar os dados e fazer os devidos tratamentos:

```
1 import pandas as pd
2
3 file_path = 'dados.xls'
4
5 data = pd.read_excel(file_path)
6
7 data['Data'] = data['Data'].astype(str)
8
9 data['Data'] = pd.to_datetime(data['Data'], format='%Y.%m')
10
11 data.set_index('Data', inplace=True)
12
13 # Dividindo a serie em treino e teste
14 train = data.iloc[:-12] # Todas as observacoes exceto as 12
    ultimas
15 test = data.iloc[-12:] # As ultimas 12 observacoes
```

No código acima, carregamos os dados do documento XLS original, que contém as colunas Data e Vendas, e dividimos a série em dados de treino e teste.

	Vendas
Data	
2003-01-01	105.95583
2003-02-01	103.86803
2003-03-01	77.94452
2003-04-01	67.33154
2003-05-01	69.07137
...	...
2024-02-01	135.38066
2024-03-01	95.83992
2024-04-01	84.85470
2024-05-01	87.60281
2024-06-01	89.50121

258 rows × 1 columns

Figura 3: Dataframe base - ST

0.2 Especificações

Para melhor especificar os dados da série, implementamos a tabela abaixo, cujos dados referem-se a:

- Data do início e do fim;
- Data de início e fim do período de treinamento;
- Data de início e fim do período de teste;
- Unidade de medida;
- Fonte de dados.

Descrição	Detalhes
Data de início e fim da série	2003-01-01 - 2024-06-01
Data de início e fim do período de treinamento	2003-01-01 - 2023-06-01
Data de início e fim do período de teste	2023-06-01 - 2024-06-01
Unidade de medida	Índice (média 2022 = 100)
Fonte de dados	IPEADATA

Tabela 1: Especificações da Série Temporal

1 - Questão 01

1.1 Questão 1: Parte (i)

Obtenha o gráfico da série no tempo, da sua 1ª diferença e da sua 2ª diferença. A partir desses gráficos determine, informalmente, o valor do parâmetro “ d ” (ordem de integração não sazonal), justificando a sua escolha. Escreva a expressão da série diferenciada utilizando o operador de atraso L , denominando essa série de z_t , sendo y_t a série original.

1.1.1 Análise Gráfica da Série Temporal e Diferenças

A série temporal original, composta por dados de vendas nominais, apresenta uma tendência clara e sazonalidade marcante, como evidenciado no gráfico. Ao aplicar a primeira e segunda diferenciação, observamos a eliminação progressiva das tendências, o que é esperado ao tentarmos tornar a série estacionária.

A visualização dos gráficos, entretanto, apresenta alguns desafios. Em comparação com plataformas como o IPEADATA, o gráfico gerado pelo Python exibe “nós” e sobreposições mais densas. Isso se deve a uma combinação de fatores:

Resolução e Tamanho do Gráfico: Os gráficos gerados em Python, por padrão, podem ter resoluções mais baixas, o que faz com que as linhas e os dados se apresentem mais “apertados”. Embora o código tenha sido ajustado para melhorar a largura das linhas e o espaçamento dos eixos, ainda existem limitações que afetam a suavidade visual.

Densidade dos Dados: A série contém um grande número de observações (anos de vendas mensais), e, ao plotar todos esses pontos de dados, é natural que os gráficos resultem em sobreposições. Em gráficos interativos ou gerados em maior resolução, esses pontos podem ser melhor explorados e analisados.

Rotação de Rótulos e Ajustes de Layout: Mesmo com a rotação de rótulos no eixo X, a densidade de dados ainda pode causar certa confusão visual em gráficos estáticos. Ferramentas interativas permitem que o observador foque em detalhes específicos, o que não é possível em gráficos estáticos como o gerado aqui.

Apesar dessas limitações, os gráficos permitem uma análise adequada da estacionaridade da série ao observar que a primeira diferenciação não elimina completamente a sazonalidade, mas torna a série mais estável. A segunda diferenciação parece eliminar mais oscilações, o que nos sugere que o valor de d poderia ser 1 ou 2, dependendo da modelagem.

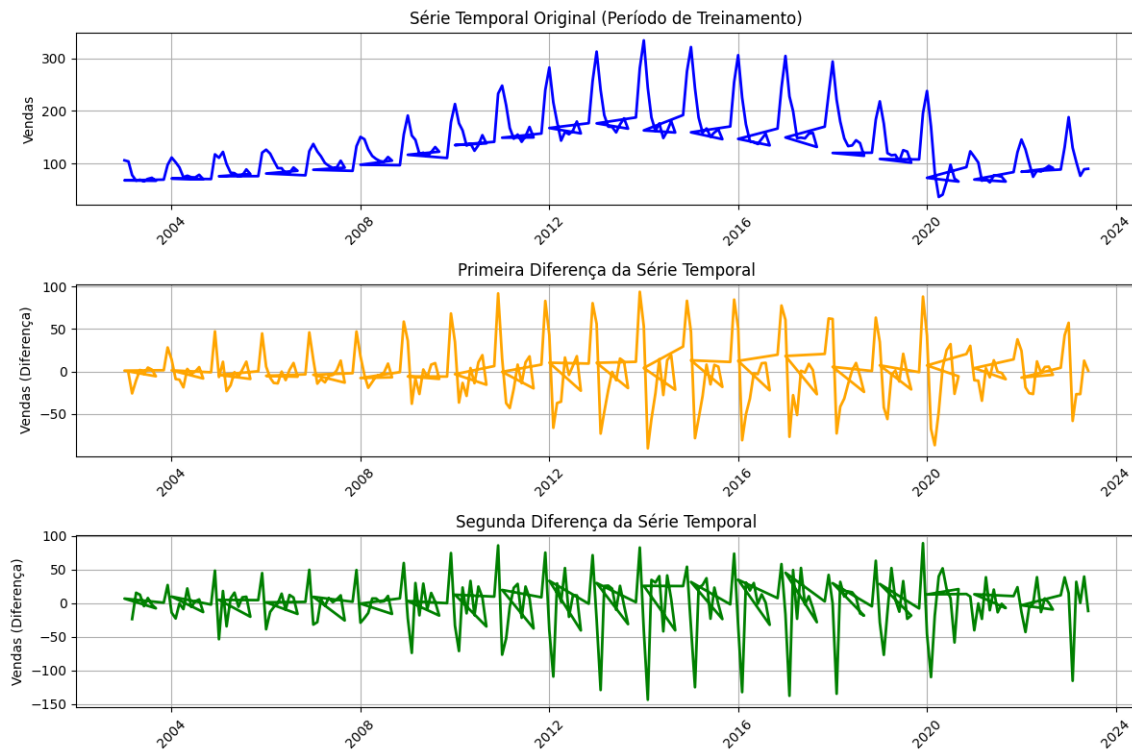


Figura 4: Gráficos plotados da série e primeira e segunda diferenças

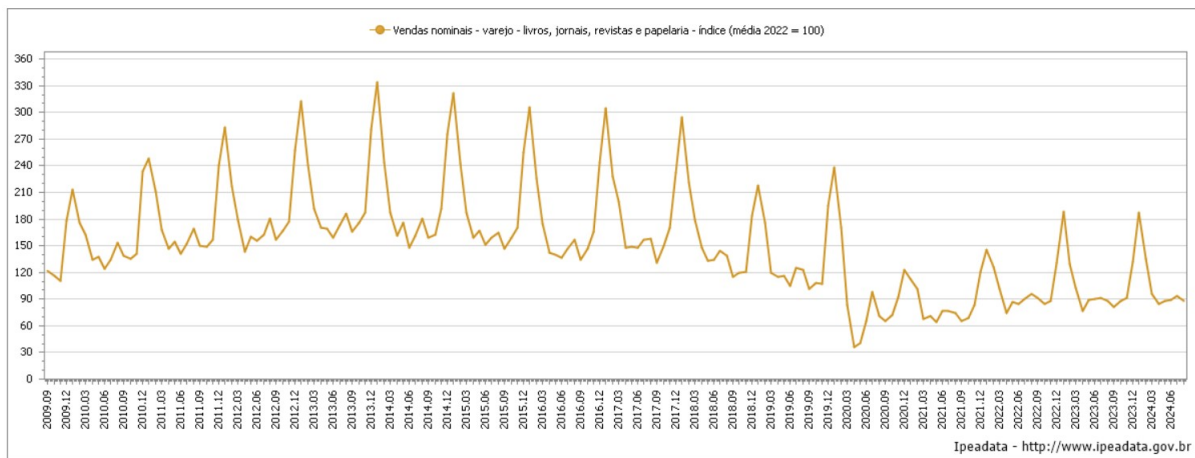


Figura 5: Gráfico do site Ipeadata

A partir da análise gráfica da série temporal original, da primeira e da segunda diferenciação, podemos determinar informalmente o valor do parâmetro d (ordem de integração não sazonal) da seguinte maneira:

- A série temporal original apresenta uma clara tendência, o que indica que a série não é estacionária. Portanto, uma diferenciação é necessária para remover essa tendência.
- Após a primeira diferenciação, observamos que a série apresenta comportamento mais estacionário, removendo a tendência, mas mantendo um padrão sazonal. Isso indica que uma única diferenciação é suficiente para remover a não-estacionaridade associada à tendência. A sazonalidade será tratada separadamente com o componente sazonal.

- Após a segunda diferenciação, a série apresenta menor variabilidade, mas não é necessário aplicar mais uma diferenciação para remover a sazonalidade.

Com base nessa análise, determinamos que o valor de d é 1, pois a primeira diferenciação já remove a tendência e torna a série estacionária em termos de tendência.

1.1.2 Expressão da Série Diferenciada

A expressão da série temporal diferenciada pode ser representada utilizando o operador de atraso L . O operador L desloca a série temporal em um período, ou seja, $L^k y_t = y_{t-k}$, onde y_t é a série original.

A primeira diferença de y_t pode ser representada como:

$$z_t = y_t - Ly_t = (1 - L)y_t$$

Portanto, a série z_t , que é a primeira diferenciação da série y_t , pode ser expressa como:

$$z_t = (1 - L)y_t$$

Como a série já se torna estacionária após a primeira diferenciação, não há necessidade de aplicar uma segunda diferenciação.

1.1.3 Justificativa do Valor de $d = 1$

Conforme a análise gráfica e os conceitos discutidos, o valor $d = 1$ é justificado pela remoção da tendência após a primeira diferenciação. A sazonalidade será tratada pelo componente sazonal do modelo SARIMA (D), evitando a necessidade de mais uma diferenciação. A ordem de integração não sazonal $d = 1$ é suficiente para modelar a série temporal, conforme observado.

1.2 Questão 1: Parte (ii)

Realizar o teste KPSS para escolher o valor da ordem de diferenciação não sazonal da série, “d”. Especificar H_0 , H_a e o resultado do teste. Comparar esse valor de “d” com aquele que vc sugerimos em i). Para esse valor de “d”, obter a série diferenciada, seguida da FAC e FACP.

1.2.1 Teste KPSS para verificar a estacionaridade da série e determinar d

O teste KPSS tem como hipótese nula H_0 que a série é estacionária, e a hipótese alternativa H_a é que a série não é estacionária.

Executamos o seguinte script em Python para realizar o teste KPSS e obter o valor d :

```
1 """ Parte (ii) """
2
3 # Realizar o teste KPSS na serie original (nao diferenciada)
4 kpss_test = kpss(train['Vendas'], regression='c', nlags='auto')
5
```

```

6 # Exibir os resultados do teste KPSS
7 print(f'KPSS Test Statistic: {kpss_test[0]}')
8 print(f'p-value: {kpss_test[1]}')
9 print(f'Critical Values: {kpss_test[3]}')
10
11 # Interpretacao
12 if kpss_test[1] < 0.05:
13     print("A serie NAO eh estacionaria (Rejeita H0)")
14 else:
15     print("A serie e estacionaria (Aceita H0)")

```

Onde obtivemos os seguintes resultados:

```

# Exibir os resultados do teste KPSS
print(f'KPSS Test Statistic: {kpss_test[0]}')
print(f'p-value: {kpss_test[1]}')
print(f'Critical Values: {kpss_test[3]}')

KPSS Test Statistic: 0.6939190882372172
p-value: 0.01409826470570753
Critical Values: {'10%': 0.347, '5%': 0.463, '2.5%': 0.574, '1%': 0.739}

if kpss_test[1] < 0.05:
    print("A série NÃO é estacionária (Rejeita H0)")
else:
    print("A série é estacionária (Aceita H0)")

A série NÃO é estacionária (Rejeita H0)

if kpss_test[1] < 0.05:
    print("A série NÃO é estacionária (Rejeita H0)")
else:
    print("A série é estacionária (Aceita H0)")

A série NÃO é estacionária (Rejeita H0)

```

Figura 6: Resultado - KPSS

1.2.2 AC (Função de Autocorrelação) e FACP (Função de Autocorrelação Parcial)

Após determinar d , o próximo passo é obter as funções de autocorrelação (FAC) e autocorrelação parcial (FACP) para entender os padrões sazonais da série.

O código para gerar os gráficos da FAC e FACP foi implementado da seguinte forma:

```

1 # Serie diferenciada com base no valor de 'd'
2 differenced_series = train['Vendas'].diff().dropna()
3
4 # Plotar a FAC e a FACP da serie diferenciada
5 plt.figure(figsize=(12, 6))
6

```

```

7 # FAC
8 plt.subplot(1, 2, 1)
9 plot_acf(differenced_series, lags=36, ax=plt.gca(),
10          title='FAC - Série Diferenciada')
11
12 # FACP
13 plt.subplot(1, 2, 2)
14 plot_pacf(differenced_series, lags=36, ax=plt.gca(),
15           title='FACP - Série Diferenciada')
16
17 plt.tight_layout()
18 plt.show()

```

Onde obtivemos os seguintes resultados:

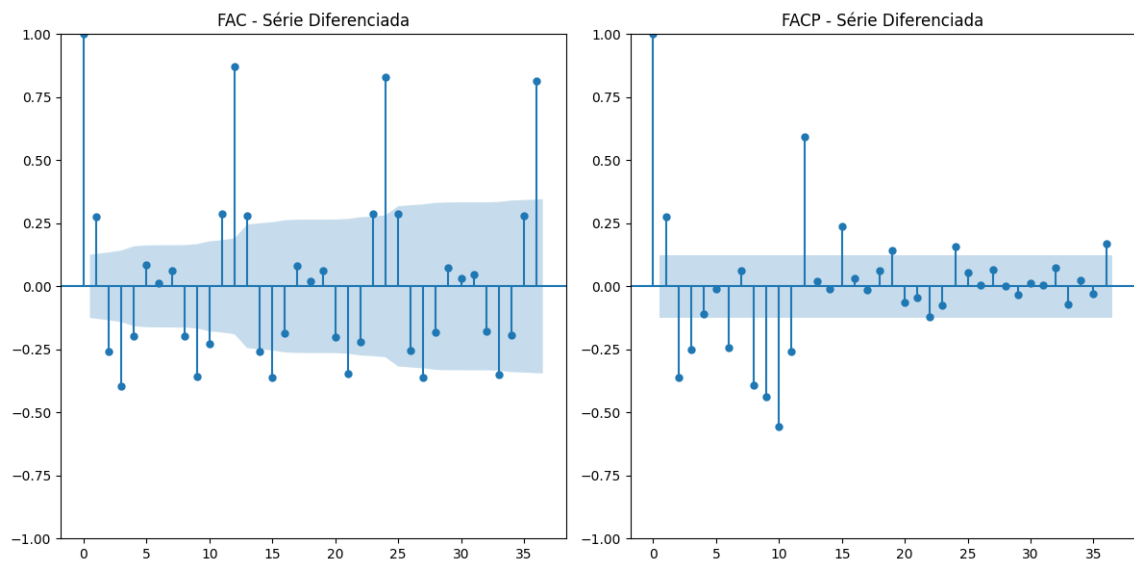


Figura 7: Gráficos da FAC e FACP

1.2.3 (a) Existe dependência sazonal nessa série?

Sim, existe dependência sazonal na série, conforme evidenciado pelos picos significativos observados na Função de Autocorrelação (FAC) nas defasagens sazonais, ou seja, múltiplos de 12 ($k = 12, 24, 36$). Esses picos indicam que existe uma correlação significativa em intervalos temporais sazonais, o que caracteriza uma forte dependência sazonal na série temporal.

1.2.4 (b) É possível sugerir um modelo ARIMA(p, d, q), de partida, para a série y_t ?

Sim, é possível sugerir um modelo ARIMA (p, d, q) de partida para a série y_t . Com base nos gráficos da Função de Autocorrelação Parcial (FACP), observam-se picos significativos nos primeiros lags, sugerindo um possível valor para p . A ausência de padrões de decaimento abrupto na FAC indica que o valor de q não precisa ser elevado. Assim, um modelo ARIMA (1, 1, 1) pode ser sugerido como um ponto de partida, sendo que $d = 1$ foi determinado pelo teste KPSS, que revelou a necessidade de uma diferenciação para tornar a série estacionária.

1.2.5 (c) O padrão de decaimento da FAC nos lags sazonais sugere a necessidade de diferenciação sazonal “D” para a série, além de “d”?

Sim, o padrão de decaimento da FAC nas lags sazonais $k = 12, 24, 36$ sugere a necessidade de uma diferenciação sazonal, ou seja, $D > 0$. Esses picos em múltiplos de 12 indicam que a série contém uma componente sazonal significativa, mesmo após a primeira diferenciação não sazonal ($d = 1$). Portanto, recomenda-se aplicar uma diferenciação sazonal de ordem $D = 1$ para capturar essa sazonalidade presente na série.

1.3 Questão 1: Parte (iii)

Obtenha o melhor “modelo não-sazonal” para a sua série original y_t . Para isso utilize, no seu software, o procedimento automático para selecionar o melhor modelo ARIMA(p,d,q) via AICc, sem incluir lags sazonais, e fixando o “d” e “D” nos valores selecionados anteriormente (vc deve informar na função que faz a busca automática que a busca não inclui lags sazonais; a função `auto.arima` do R tem essa possibilidade).

Nesta parte, vamos começar usando a função `"auto_arima"` da biblioteca *pmdarima*, que suporta a seleção de modelos ARIMA com base no AICc. Precisamos nos certificar também de ajustar para que não seja considerada a sazonalidade.

Para encontrar o melhor modelo não-sazonal para a série y_t , utilizamos o procedimento de busca automática pelo critério AICc, conforme solicitado. Para isso, utilizamos o código Python baseado na função `auto_arima` da biblioteca *pmdarima*, que permite ajustar modelos ARIMA automaticamente com base em diversos critérios de informação, como o AICc.

O código utilizado foi o seguinte:

```
1 import pmdarima as pm
2
3 """ Parte iii """
4 # Ajustar o modelo ARIMA nao-sazonal com base no AICc
5 model = pm.auto_arima(train['Vendas'],
6                       start_p=0, max_p=5,
7                       start_q=0, max_q=5,
8                       d=1, #d foi escolhido antes como 1
9                       seasonal=False, #Ignorar sazonalidade
10                      information_criterion='aicc', #
11                      Criterio AICc
12                      trace=True,
13                      stepwise=True)
14 # Exibindo o resumo do modelo
15 print(model.summary())
16
17 # Valor de AICc
18 aicc_value = model.aicc()
```

```
19 print(f"Valor do AICc: {aicc_value}")
```

Esse código realiza o ajuste do modelo ARIMA não-sazonal, com $d = 1$ definido previamente e a busca pelos valores de p e q variando de 0 a 5. O critério de seleção foi o AICc, que penaliza modelos mais complexos, sendo ideal para a escolha de modelos de séries temporais. O código também ignorou a sazonalidade (`seasonal=False`), como especificado na questão. O valor de $d = 1$ foi escolhido com base na análise anterior (parte i), onde determinamos que a série precisava de uma diferenciação não-sazonal.

```
Performing stepwise search to minimize aicc
ARIMA(0,1,0)(0,0,0)[0] intercept : AICC=2388.242, Time=0.52 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AICC=2370.930, Time=0.06 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AICC=2357.891, Time=0.17 sec
ARIMA(0,1,0)(0,0,0)[0] intercept : AICC=2386.211, Time=0.01 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : AICC=2358.460, Time=0.19 sec
ARIMA(0,1,2)(0,0,0)[0] intercept : AICC=2340.579, Time=0.19 sec
ARIMA(1,1,2)(0,0,0)[0] intercept : AICC=2312.557, Time=0.20 sec
ARIMA(2,1,2)(0,0,0)[0] intercept : AICC=2295.125, Time=0.22 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : AICC=2293.048, Time=0.14 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AICC=2338.949, Time=0.08 sec
ARIMA(3,1,1)(0,0,0)[0] intercept : AICC=2295.125, Time=0.24 sec
ARIMA(3,1,0)(0,0,0)[0] intercept : AICC=2325.354, Time=0.14 sec
ARIMA(3,1,2)(0,0,0)[0] intercept : AICC=2294.938, Time=0.45 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : AICC=2291.018, Time=0.07 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : AICC=2356.394, Time=0.05 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AICC=2336.882, Time=0.05 sec
ARIMA(3,1,1)(0,0,0)[0] intercept : AICC=2293.077, Time=0.19 sec
ARIMA(2,1,2)(0,0,0)[0] intercept : AICC=2293.077, Time=0.10 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AICC=2368.881, Time=0.02 sec
ARIMA(1,1,2)(0,0,0)[0] intercept : AICC=2310.498, Time=0.11 sec
ARIMA(3,1,0)(0,0,0)[0] intercept : AICC=2323.270, Time=0.09 sec
ARIMA(3,1,2)(0,0,0)[0] intercept : AICC=2292.868, Time=0.17 sec

Best model: ARIMA(2,1,1)(0,0,0)[0]
Total fit time: 3.474 seconds
```

Figura 8: Performing stepwise search to minimize aicc

1.3.1 (a) Melhor modelo e valor do AICc

Após rodar o código, o modelo selecionado foi um **ARIMA(2,1,1)**, conforme a tabela de resultados gerada:

SARIMAX Results					
=====					
Dep. Variable:	y	No. Observations:	246		
Model:	SARIMAX(2, 1, 1)	Log Likelihood	-1141.426		
Date:	Sat, 19 Oct 2024	AIC	2290.852		
Time:	15:43:41	BIC	2304.857		
Sample:	0	HQIC	2296.492		
	- 246				
Covariance Type:	opg				
=====					
	coef	std err	z	P> z	[0.025 0.975]

ar.L1	1.0038	0.049	20.408	0.000	0.907 1.100
ar.L2	-0.5265	0.061	-8.593	0.000	-0.647 -0.406
ma.L1	-0.8866	0.039	-22.743	0.000	-0.963 -0.810
sigma2	648.0274	45.701	14.180	0.000	558.455 737.600
=====					
Ljung-Box (L1) (Q):	0.01	Jarque-Bera (JB):	59.92		
Prob(Q):	0.93	Prob(JB):	0.00		
Heteroskedasticity (H):	3.49	Skew:	0.85		
Prob(H) (two-sided):	0.00	Kurtosis:	4.73		
=====					
Warnings:					
[1] Covariance matrix calculated using the outer product of gradients (complex-step).					
Valor do AICc: 2291.01788968608					

Figura 9: Tabela de resultados - SARIMA

$$\text{SARIMA}(2, 1, 1)(0, 0, 0)_{12}$$

A equação SARIMA correspondente ao modelo ajustado é a seguinte:

$$(1 - 1.0038L + 0.5265L^2)(1 - L)y_t = (1 + 0.8866L)\epsilon_t$$

Neste caso:

- $p = 2$: O modelo ARIMA selecionou dois termos autorregressivos.
- $d = 1$: Foi aplicada uma única diferenciação para tornar a série estacionária.
- $q = 1$: Um termo de média móvel foi selecionado.
- $P = 0, D = 0, Q = 0$: Não houve inclusão de componentes sazonais.

O valor do **AICc** foi **2291.01789**, indicando o melhor modelo ajustado segundo esse critério.

1.3.2 (b) Análise da FAC e FACP dos resíduos

Após o ajuste do modelo ARIMA, obtivemos os resíduos do modelo para analisar se restaram padrões de dependência não explicados. O código utilizado para calcular a FAC e FACP dos resíduos foi:

```

1 # Obter os res duos do modelo ajustado
2 residuals = model.resid()
3

```



```

4 # Plotar a FAC e FACP dos res duos
5 plt.figure(figsize=(12, 6))
6
7 # FAC dos res duos
8 plt.subplot(1, 2, 1)
9 plot_acf(residuals, lags=36, ax=plt.gca(), title='FAC dos
    Res duos')
10
11 # FACP dos res duos
12 plt.subplot(1, 2, 2)
13 plot_pacf(residuals, lags=36, ax=plt.gca(), title='FACP dos
    Res duos')
14
15 plt.tight_layout()
16 plt.show()

```

Os gráficos gerados (FAC e FACP) nos ajudam a observar se ainda existem padrões remanescentes de correlação nos resíduos, tanto em lags de curto prazo quanto em lags sazonais.

Resultados da FAC e FACP dos Resíduos:

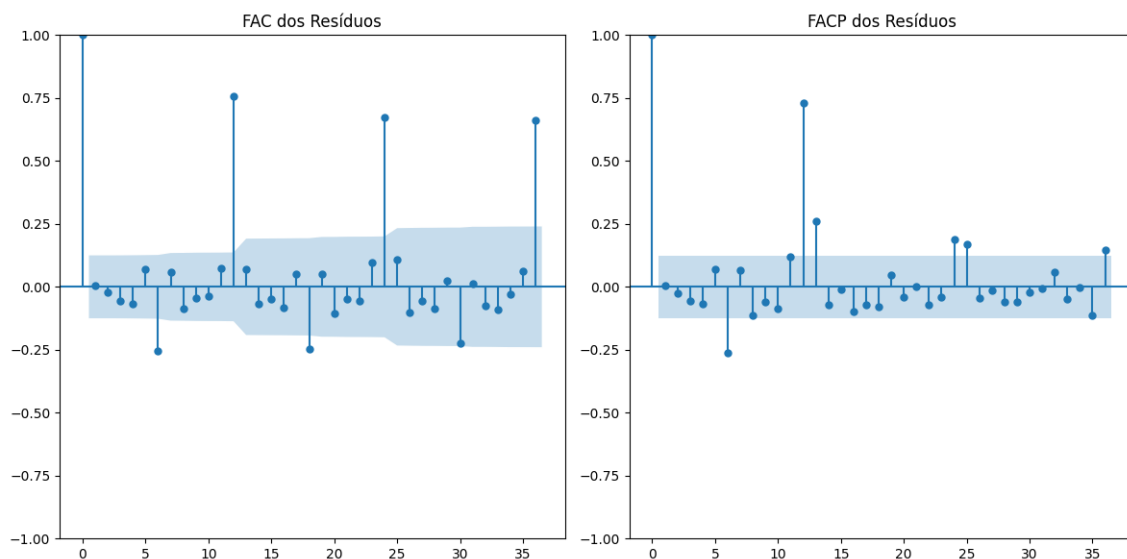


Figura 10: Gráficos da FAC e da FACP dos resíduos

A seguir, analisamos os valores da Função de Autocorrelação (FAC) e da Função de Autocorrelação Parcial (FACP) dos resíduos do modelo ajustado, com foco nos lags de curto prazo $k = 1, 2, 3, 4$ e no lag sazonal $k = 12$, conforme solicitado.

FAC dos Resíduos

- Para os lags de curto prazo $k = 1, 2, 3, 4$, a FAC dos resíduos mostra que os valores estão todos dentro dos limites de confiança. Isso indica que o modelo ARIMA(2,1,1)

foi eficaz em capturar as correlações de curto prazo da série, não deixando dependências remanescentes nesses lags.

- No entanto, para o lag sazonal $k = 12$, observamos um pico significativo fora dos limites de confiança, sugerindo que o modelo não conseguiu capturar adequadamente a dependência sazonal presente na série. Além disso, um pico similar ocorre no lag $k = 24$, o que pode reforçar a ideia de uma dependência sazonal residual.

FACP dos Resíduos

- Nos lags de curto prazo $k = 1, 2, 3, 4$, a FACP dos resíduos também está dentro dos limites de confiança, confirmando que o modelo capturou bem a estrutura de curto prazo da série temporal.
- No lag sazonal $k = 12$, a FACP mostra um pico similar ao observado na FAC, sugerindo que o modelo ARIMA(2,1,1) não foi capaz de capturar completamente a sazonalidade presente na série.

Com base na análise dos resíduos, concluímos que o modelo ARIMA(2,1,1) conseguiu capturar bem as dependências de curto prazo, mas deixou uma dependência sazonal remanescente, como indicado pelos picos em $k = 12$ e $k = 24$. Isso sugere que, embora o modelo ajustado seja adequado para capturar as tendências de curto prazo, a inclusão de componentes sazonais pode ser necessária para capturar completamente a estrutura da série temporal.

1.4 Questão 1: Parte (iv)

Nesta parte, ajustamos um novo modelo SARIMA com sazonalidade habilitada, utilizando o mesmo procedimento automático mencionado na parte (iii), mas agora permitindo que o procedimento escolha automaticamente os valores de d (ordem de integração não-sazonal) e D (ordem de diferenciação sazonal). O modelo resultante é denominado sub-ótimo.

```
1 """ Parte iv """
2 # Ajustar o modelo SARIMA com sazonalidade habilitada
3 model_sarima = pm.auto_arima(train['Vendas'],
4                               start_p=0, max_p=5,
5                               start_q=0, max_q=5,
6                               d=None, D=None,      # Permitir
7                               que o modelo escolha d e D
8                               seasonal=True,      # Habilitar
9                               sazonalidade
10                               m=12,              # Per odo
11                               sazonal (12 meses)
12                               information_criterion='aicc',
13                               trace=True,
14                               stepwise=True)
15
16 # Exibir o resumo do modelo
17 print(model_sarima.summary())
```

```

15
16 # Valor de AICc
17 aicc_value_sarima = model_sarima.aicc()
18 print(f"Valor do AICc: {aicc_value_sarima}")
19
20 # Obter os res duos do modelo ajustado
21 residuals_sarima = model_sarima.resid()

```

O código acima executa a busca stepwise por meio da função `auto_arima` da biblioteca `pmdarima`. O procedimento tenta várias combinações de valores para os parâmetros p , q , P , Q , incluindo a parte sazonal e o valor de $m = 12$, que se refere à sazonalidade mensal da série. O objetivo é minimizar o critério de informação AICc, selecionando o modelo que melhor equilibra ajuste e complexidade, de acordo com o princípio da parcimônia. O processo automático testou mais de 80 combinações diferentes de parâmetros, como mostrado na saída abaixo:

```

Performing stepwise search to minimize aicc
ARIMA(0,1,0)(1,0,1)[12] intercept : AICC=1987.874, Time=0.33 sec
ARIMA(0,1,0)(0,0,0)[12] intercept : AICC=2388.242, Time=0.01 sec
ARIMA(1,1,0)(1,0,0)[12] intercept : AICC=2014.585, Time=0.17 sec
ARIMA(0,1,1)(0,0,1)[12] intercept : AICC=2198.753, Time=0.15 sec
ARIMA(0,1,0)(0,0,0)[12] : AICC=2386.211, Time=0.01 sec
ARIMA(0,1,0)(0,0,1)[12] intercept : AICC=2211.816, Time=0.13 sec
ARIMA(0,1,0)(1,0,0)[12] intercept : AICC=2014.213, Time=0.14 sec
ARIMA(0,1,0)(2,0,1)[12] intercept : AICC=1989.862, Time=0.76 sec
ARIMA(0,1,0)(1,0,2)[12] intercept : AICC=1989.777, Time=0.49 sec
ARIMA(0,1,0)(0,0,2)[12] intercept : AICC=2154.849, Time=0.35 sec
ARIMA(0,1,0)(2,0,0)[12] intercept : AICC=inf, Time=0.45 sec
ARIMA(0,1,0)(2,0,2)[12] intercept : AICC=inf, Time=1.61 sec
ARIMA(1,1,0)(1,0,1)[12] intercept : AICC=1987.505, Time=0.31 sec
ARIMA(1,1,0)(0,0,1)[12] intercept : AICC=2203.781, Time=0.20 sec
ARIMA(1,1,0)(2,0,1)[12] intercept : AICC=1989.322, Time=1.38 sec
ARIMA(1,1,0)(1,0,2)[12] intercept : AICC=1989.037, Time=0.77 sec
ARIMA(1,1,0)(0,0,0)[12] intercept : AICC=2370.930, Time=0.04 sec
ARIMA(1,1,0)(0,0,2)[12] intercept : AICC=2154.665, Time=0.57 sec
ARIMA(1,1,0)(2,0,0)[12] intercept : AICC=inf, Time=2.67 sec
ARIMA(1,1,0)(2,0,2)[12] intercept : AICC=1987.629, Time=2.66 sec
ARIMA(2,1,0)(1,0,1)[12] intercept : AICC=1986.706, Time=0.43 sec
ARIMA(2,1,0)(0,0,1)[12] intercept : AICC=2186.890, Time=0.23 sec
ARIMA(2,1,0)(1,0,0)[12] intercept : AICC=2013.674, Time=0.30 sec
ARIMA(2,1,0)(2,0,1)[12] intercept : AICC=1988.519, Time=1.32 sec
ARIMA(2,1,0)(1,0,2)[12] intercept : AICC=1988.246, Time=0.84 sec
ARIMA(2,1,0)(0,0,0)[12] intercept : AICC=2338.949, Time=0.06 sec
ARIMA(2,1,0)(0,0,2)[12] intercept : AICC=2147.087, Time=0.78 sec
ARIMA(2,1,0)(2,0,0)[12] intercept : AICC=inf, Time=0.68 sec
ARIMA(2,1,0)(2,0,2)[12] intercept : AICC=1987.640, Time=1.29 sec
ARIMA(3,1,0)(1,0,1)[12] intercept : AICC=1967.095, Time=0.47 sec
ARIMA(3,1,0)(0,0,1)[12] intercept : AICC=2167.358, Time=0.80 sec
ARIMA(3,1,0)(1,0,0)[12] intercept : AICC=1989.856, Time=1.76 sec
ARIMA(3,1,0)(2,0,1)[12] intercept : AICC=1969.131, Time=1.71 sec

```

ARIMA(3,1,0)(1,0,2)[12]	intercept	: AICC=1969.057, Time=1.00 sec
ARIMA(3,1,0)(0,0,0)[12]	intercept	: AICC=2325.354, Time=0.11 sec
ARIMA(3,1,0)(0,0,2)[12]	intercept	: AICC=2119.265, Time=0.56 sec
ARIMA(3,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=2.26 sec
ARIMA(3,1,0)(2,0,2)[12]	intercept	: AICC=1969.278, Time=5.00 sec
ARIMA(4,1,0)(1,0,1)[12]	intercept	: AICC=1964.701, Time=0.58 sec
ARIMA(4,1,0)(0,0,1)[12]	intercept	: AICC=2165.654, Time=0.25 sec
ARIMA(4,1,0)(1,0,0)[12]	intercept	: AICC=1986.796, Time=0.45 sec
ARIMA(4,1,0)(2,0,1)[12]	intercept	: AICC=1966.769, Time=2.06 sec
ARIMA(4,1,0)(1,0,2)[12]	intercept	: AICC=1966.694, Time=1.31 sec
ARIMA(4,1,0)(0,0,0)[12]	intercept	: AICC=2324.469, Time=0.19 sec
ARIMA(4,1,0)(0,0,2)[12]	intercept	: AICC=2118.504, Time=0.66 sec
ARIMA(4,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=1.21 sec
ARIMA(4,1,0)(2,0,2)[12]	intercept	: AICC=1966.749, Time=2.34 sec
ARIMA(5,1,0)(1,0,1)[12]	intercept	: AICC=1966.850, Time=0.78 sec
ARIMA(4,1,1)(1,0,1)[12]	intercept	: AICC=1960.430, Time=0.67 sec
ARIMA(4,1,1)(0,0,1)[12]	intercept	: AICC=2164.477, Time=0.70 sec
ARIMA(4,1,1)(1,0,0)[12]	intercept	: AICC=1980.002, Time=0.84 sec
ARIMA(4,1,1)(2,0,1)[12]	intercept	: AICC=1962.595, Time=1.96 sec
ARIMA(4,1,1)(1,0,2)[12]	intercept	: AICC=1962.589, Time=1.55 sec
ARIMA(4,1,1)(0,0,0)[12]	intercept	: AICC=2318.835, Time=0.25 sec
ARIMA(4,1,1)(0,0,2)[12]	intercept	: AICC=2097.331, Time=1.45 sec
ARIMA(4,1,1)(2,0,0)[12]	intercept	: AICC=1964.589, Time=1.29 sec
ARIMA(4,1,1)(2,0,2)[12]	intercept	: AICC=1963.506, Time=2.23 sec
ARIMA(3,1,1)(1,0,1)[12]	intercept	: AICC=1959.868, Time=0.52 sec
ARIMA(3,1,1)(0,0,1)[12]	intercept	: AICC=2142.482, Time=0.46 sec
ARIMA(3,1,1)(1,0,0)[12]	intercept	: AICC=1981.857, Time=0.60 sec
ARIMA(3,1,1)(2,0,1)[12]	intercept	: AICC=1962.001, Time=1.74 sec
ARIMA(3,1,1)(1,0,2)[12]	intercept	: AICC=1961.984, Time=1.24 sec
ARIMA(3,1,1)(0,0,0)[12]	intercept	: AICC=2295.125, Time=0.24 sec
ARIMA(3,1,1)(0,0,2)[12]	intercept	: AICC=2097.984, Time=0.86 sec
ARIMA(3,1,1)(2,0,0)[12]	intercept	: AICC=1964.230, Time=2.02 sec
ARIMA(3,1,1)(2,0,2)[12]	intercept	: AICC=1962.657, Time=2.02 sec
ARIMA(2,1,1)(1,0,1)[12]	intercept	: AICC=1963.375, Time=0.49 sec
ARIMA(3,1,2)(1,0,1)[12]	intercept	: AICC=1961.199, Time=0.82 sec
ARIMA(2,1,2)(1,0,1)[12]	intercept	: AICC=1963.114, Time=0.77 sec
ARIMA(4,1,2)(1,0,1)[12]	intercept	: AICC=inf, Time=1.17 sec
ARIMA(3,1,1)(1,0,1)[12]		: AICC=1957.756, Time=0.37 sec
ARIMA(3,1,1)(0,0,1)[12]		: AICC=2140.411, Time=0.34 sec
ARIMA(3,1,1)(1,0,0)[12]		: AICC=1979.765, Time=0.36 sec
ARIMA(3,1,1)(2,0,1)[12]		: AICC=1959.872, Time=1.14 sec
ARIMA(3,1,1)(1,0,2)[12]		: AICC=1959.855, Time=1.01 sec
ARIMA(3,1,1)(0,0,0)[12]		: AICC=2293.077, Time=0.13 sec
ARIMA(3,1,1)(0,0,2)[12]		: AICC=2095.968, Time=0.73 sec
ARIMA(3,1,1)(2,0,0)[12]		: AICC=1962.127, Time=0.65 sec
ARIMA(3,1,1)(2,0,2)[12]		: AICC=1960.523, Time=1.77 sec
ARIMA(2,1,1)(1,0,1)[12]		: AICC=1961.284, Time=0.47 sec
ARIMA(3,1,0)(1,0,1)[12]		: AICC=1964.981, Time=0.34 sec
ARIMA(4,1,1)(1,0,1)[12]		: AICC=1958.302, Time=0.55 sec
ARIMA(3,1,2)(1,0,1)[12]		: AICC=1959.071, Time=0.53 sec

```

ARIMA(2,1,0)(1,0,1)[12]           : AICC=1984.607, Time=0.22 sec
ARIMA(2,1,2)(1,0,1)[12]           : AICC=1961.003, Time=0.63 sec
ARIMA(4,1,0)(1,0,1)[12]           : AICC=1962.572, Time=0.41 sec
ARIMA(4,1,2)(1,0,1)[12]           : AICC=inf, Time=1.10 sec

```

Best model: ARIMA(3,1,1)(1,0,1)[12]

Total fit time: 77.364 seconds

SARIMAX Results						
Dep. Variable:	y			No. Observations:	246	
Model:	SARIMAX(3, 1, 1)x(1, 0, 1, 12)			Log Likelihood	-971.643	
Date:	Sat, 19 Oct 2024			AIC	1957.285	
Time:	16:44:42			BIC	1981.794	
Sample:	0			HQIC	1967.155	
	- 246					
Covariance Type:	opg					
	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.5061	0.100	5.078	0.000	0.311	0.701
ar.L2	-0.0661	0.057	-1.167	0.243	-0.177	0.045
ar.L3	-0.1816	0.067	-2.721	0.007	-0.312	-0.051
ma.L1	-0.7308	0.088	-8.313	0.000	-0.903	-0.559
ar.S.L12	0.9451	0.015	61.802	0.000	0.915	0.975
ma.S.L12	-0.3697	0.058	-6.357	0.000	-0.484	-0.256
sigma2	150.9869	9.912	15.232	0.000	131.559	170.415
Ljung-Box (L1) (Q):	0.03		Jarque-Bera (JB):	60.78		
Prob(Q):	0.86		Prob(JB):	0.00		
Heteroskedasticity (H):	5.49		Skew:	-0.26		
Prob(H) (two-sided):	0.00		Kurtosis:	5.38		

Figura 11: Tabela de resultados - SARIMA

1.4.1 (a) Quantos modelos foram comparados para selecionar o modelo sub-ótimo?

O procedimento `auto_arima` comparou **87 modelos** durante a busca stepwise, considerando várias combinações de valores para p , q , P , Q e incluindo sazonalidade com $m = 12$. O melhor modelo selecionado foi o **SARIMA(3,1,1)(1,0,1)₁₂**, que minimizou o critério AICc, tornando-se o modelo sub-ótimo.

1.4.2 (b) AICc e a equação SARIMA(p,d,q)(P,D,Q) em termos das ordens p, q, d, D, P e Q

O melhor modelo SARIMA selecionado foi o **SARIMA(3,1,1)(1,0,1)₁₂**, que inclui tanto componentes não-sazonais quanto componentes sazonais com período $m = 12$. A tabela com os parâmetros do modelo foi gerada pelo procedimento `auto_arima` e está resumida a seguir:

- $p = 3$: três termos autorregressivos não-sazonais;

- $d = 1$: uma diferenciação não-sazonal;
- $q = 1$: um termo de média móvel não-sazonal;
- $P = 1$: um termo autorregressivo sazonal com período $m = 12$;
- $D = 0$: nenhuma diferenciação sazonal;
- $Q = 1$: um termo de média móvel sazonal com período $m = 12$.

O valor do critério de informação AICc para esse modelo foi 1957.756. Esse critério sugere que o modelo é o mais parcimonioso entre os testados, balanceando ajuste e simplicidade. A equação SARIMA correspondente ao modelo ajustado é:

$$(1 - 0.5061L + 0.0661L^2 + 0.1816L^3)(1 - L)y_t = (1 - 0.7308L)(1 - 0.9451L^{12} + 0.3697L^{12})\epsilon_t$$

Onde:

- y_t é a série temporal original;
- L é o operador de defasagem;
- ϵ_t é o erro aleatório (ruído branco).

1.4.3 (c) FAC e FACP dos resíduos e o resultado do teste Ljung-Box para $m = 20$

Após o ajuste do modelo, analisamos a Função de Autocorrelação (FAC) e a Função de Autocorrelação Parcial (FACP) dos resíduos para verificar a presença de padrões de dependência não capturados pelo modelo. Além disso, realizamos o teste de Ljung-Box com $m = 20$, que testa a hipótese de que não há autocorrelação significativa até a defasagem 20.

O código para gerar os gráficos da FAC e FACP e realizar o teste de Ljung-Box é mostrado a seguir:

```

1 # Obter os res duos do modelo ajustado
2 residuals_sarima = model_sarima.resid()
3
4 # Plotar a FAC e FACP dos res duos
5 plt.figure(figsize=(12, 6))
6
7 # FAC dos res duos
8 plt.subplot(1, 2, 1)
9 plot_acf(residuals_sarima, lags=36, ax=plt.gca(),
10          title='FAC dos Res duos (SARIMA)')
11
12 # FACP dos res duos
13 plt.subplot(1, 2, 2)
14 plot_pacf(residuals_sarima, lags=36, ax=plt.gca(),
15           title='FACP dos Res duos (SARIMA)')
16
17 plt.tight_layout()
18 plt.show()

```

```

19
20 # Teste de Ljung-Box para m=20
21 ljung_box_results = acorr_ljungbox(residuals_sarima, lags
22                                   =[20], return_df=True)
23 print(ljung_box_results)

```

FAC e FACP dos Resíduos

Os gráficos da Função de Autocorrelação (FAC) e da Função de Autocorrelação Parcial (FACP) dos resíduos são mostrados abaixo:

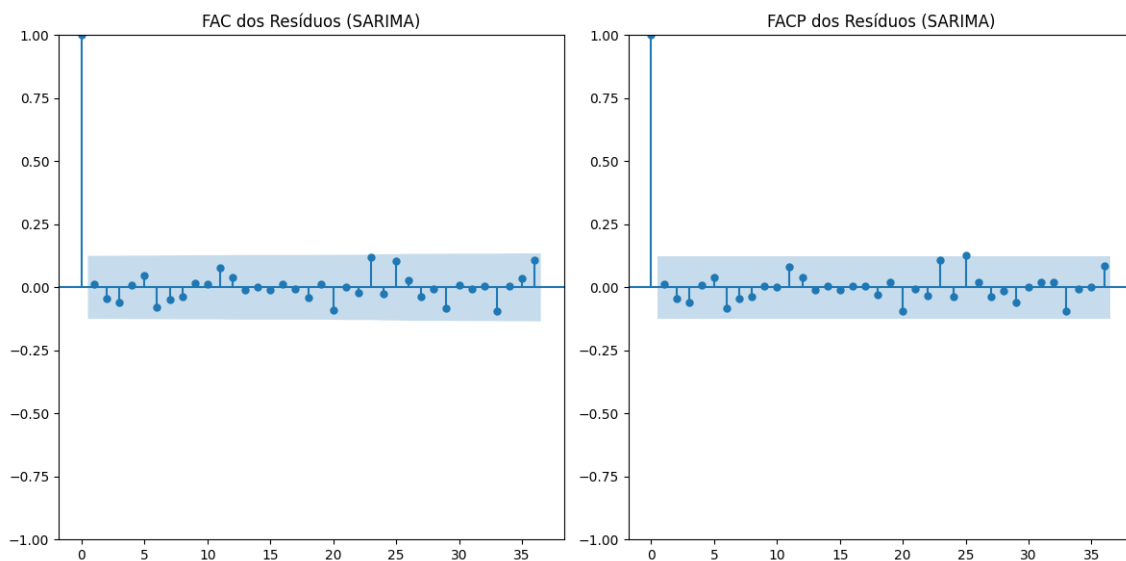


Figura 12: Gráficos da FAC e da FACP dos resíduos do modelo SARIMA(3,1,1)(1,0,1)₁₂

- A **FAC dos resíduos** mostra que os resíduos do modelo estão dentro dos limites de confiança para todas as defasagens, sugerindo que o modelo capturou adequadamente a estrutura de dependência da série temporal.
- A **FACP dos resíduos** confirma essa análise, já que nenhum lag mostra correlação significativa.

Resultado do Teste de Ljung-Box para $m = 20$

O teste de Ljung-Box foi aplicado para verificar a independência dos resíduos até a defasagem $m = 20$, e os resultados foram os seguintes:

$$\text{Estatística Ljung-Box} = 9.26935, \quad \text{p-valor} = 0.979572$$

Como o p-valor é significativamente maior que 0.05, não podemos rejeitar a hipótese nula de que os resíduos são independentes. Isso indica que não há autocorrelação significativa nos resíduos do modelo SARIMA(3,1,1)(1,0,1)₁₂, sugerindo que o modelo é adequado e os resíduos se comportam como ruído branco.

Com base nesses resultados, concluímos que o modelo sub-ótimo SARIMA(3,1,1)(1,0,1)₁₂ é eficaz para capturar a estrutura da série temporal, não deixando padrões de dependência não explicados nos resíduos.

1.5 Questão 1: Parte (v)

Sabemos que as funções de ajuste automático do SARIMA geralmente realizam uma busca “econômica” para poupar recursos computacionais. Agora queremos que vc realize uma busca mais extensa para chegar ao melhor modelo SARIMA para a sua série (denomine esse modelo de ótimo).

Nesta primeira etapa, podemos utilizar o *auto_arima* com uma busca mais extensa, removendo a opção de busca "stepwise", que é mais econômica computacionalmente. Isso permitirá que o algoritmo teste um número maior de combinações de parâmetros.

O código foi ajustado da seguinte forma:

```
1 # Ajustar o modelo SARIMA com uma busca mais extensa (sem
   stepwise)
2 model_sarima_extenso = pm.auto_arima(train['Vendas'],
3                                     start_p=0, max_p=5,
4                                     start_q=0, max_q=5,
5                                     d=None, D=None,      #Permitir que o modelo
                                     escolha d e D
6                                     seasonal=True,      #Habilitar sazonalidade
7                                     # Período sazonal (12 meses)
8                                     m=12,
9                                     information_criterion='aicc',
10                                    trace=True,
11                                    stepwise=False)      #Desabilitar busca
                                     stepwise
12
13 # Exibir o resumo do modelo
14 print(model_sarima_extenso.summary())
15
16 # Valor de AICc
17 aicc_value_extenso = model_sarima_extenso.aicc()
18 print(f"Valor do AICc: {aicc_value_extenso}")
```

```
ARIMA(0,1,0)(0,0,0)[12] intercept : AICC=2388.242, Time=0.02 sec
ARIMA(0,1,0)(0,0,1)[12] intercept : AICC=2211.816, Time=0.17 sec
ARIMA(0,1,0)(0,0,2)[12] intercept : AICC=2154.849, Time=0.31 sec
ARIMA(0,1,0)(1,0,0)[12] intercept : AICC=2014.213, Time=0.17 sec
ARIMA(0,1,0)(1,0,1)[12] intercept : AICC=1987.874, Time=0.25 sec
ARIMA(0,1,0)(1,0,2)[12] intercept : AICC=1989.777, Time=0.87 sec
ARIMA(0,1,0)(2,0,0)[12] intercept : AICC=inf, Time=0.61 sec
```


ARIMA(0,1,0)(2,0,1)[12] intercept : AICC=1989.862, Time=0.66 sec
 ARIMA(0,1,0)(2,0,2)[12] intercept : AICC=inf, Time=3.47 sec
 ARIMA(0,1,1)(0,0,0)[12] intercept : AICC=2357.891, Time=0.07 sec
 ARIMA(0,1,1)(0,0,1)[12] intercept : AICC=2198.753, Time=0.16 sec
 ARIMA(0,1,1)(0,0,2)[12] intercept : AICC=2153.700, Time=0.50 sec
 ARIMA(0,1,1)(1,0,0)[12] intercept : AICC=2013.866, Time=0.21 sec
 ARIMA(0,1,1)(1,0,1)[12] intercept : AICC=1986.478, Time=0.31 sec
 ARIMA(0,1,1)(1,0,2)[12] intercept : AICC=1987.732, Time=0.81 sec
 ARIMA(0,1,1)(2,0,0)[12] intercept : AICC=1990.705, Time=0.58 sec
 ARIMA(0,1,1)(2,0,1)[12] intercept : AICC=1988.166, Time=0.70 sec
 ARIMA(0,1,1)(2,0,2)[12] intercept : AICC=1985.975, Time=1.71 sec
 ARIMA(0,1,2)(0,0,0)[12] intercept : AICC=2340.579, Time=0.09 sec
 ARIMA(0,1,2)(0,0,1)[12] intercept : AICC=2185.960, Time=0.22 sec
 ARIMA(0,1,2)(0,0,2)[12] intercept : AICC=2136.121, Time=0.74 sec
 ARIMA(0,1,2)(1,0,0)[12] intercept : AICC=1999.933, Time=0.32 sec
 ARIMA(0,1,2)(1,0,1)[12] intercept : AICC=1974.432, Time=0.42 sec
 ARIMA(0,1,2)(1,0,2)[12] intercept : AICC=1975.738, Time=1.80 sec
 ARIMA(0,1,2)(2,0,0)[12] intercept : AICC=1977.315, Time=1.59 sec
 ARIMA(0,1,2)(2,0,1)[12] intercept : AICC=1976.081, Time=0.98 sec
 ARIMA(0,1,3)(0,0,0)[12] intercept : AICC=2305.408, Time=0.38 sec
 ARIMA(0,1,3)(0,0,1)[12] intercept : AICC=2149.221, Time=0.34 sec
 ARIMA(0,1,3)(0,0,2)[12] intercept : AICC=2095.536, Time=1.47 sec
 ARIMA(0,1,3)(1,0,0)[12] intercept : AICC=1975.217, Time=0.31 sec
 ARIMA(0,1,3)(1,0,1)[12] intercept : AICC=1954.929, Time=0.56 sec
 ARIMA(0,1,3)(2,0,0)[12] intercept : AICC=1959.068, Time=0.86 sec
 ARIMA(0,1,4)(0,0,0)[12] intercept : AICC=2293.762, Time=0.14 sec
 ARIMA(0,1,4)(0,0,1)[12] intercept : AICC=2140.649, Time=0.50 sec
 ARIMA(0,1,4)(1,0,0)[12] intercept : AICC=1976.517, Time=0.71 sec
 ARIMA(0,1,5)(0,0,0)[12] intercept : AICC=2282.101, Time=0.46 sec
 ARIMA(1,1,0)(0,0,0)[12] intercept : AICC=2370.930, Time=0.04 sec
 ARIMA(1,1,0)(0,0,1)[12] intercept : AICC=2203.781, Time=0.25 sec
 ARIMA(1,1,0)(0,0,2)[12] intercept : AICC=2154.665, Time=0.74 sec
 ARIMA(1,1,0)(1,0,0)[12] intercept : AICC=2014.585, Time=0.23 sec
 ARIMA(1,1,0)(1,0,1)[12] intercept : AICC=1987.505, Time=0.32 sec
 ARIMA(1,1,0)(1,0,2)[12] intercept : AICC=1989.037, Time=1.02 sec
 ARIMA(1,1,0)(2,0,0)[12] intercept : AICC=inf, Time=0.61 sec
 ARIMA(1,1,0)(2,0,1)[12] intercept : AICC=1989.322, Time=0.68 sec
 ARIMA(1,1,0)(2,0,2)[12] intercept : AICC=1987.629, Time=1.26 sec
 ARIMA(1,1,1)(0,0,0)[12] intercept : AICC=2358.460, Time=0.12 sec
 ARIMA(1,1,1)(0,0,1)[12] intercept : AICC=2200.250, Time=0.45 sec
 ARIMA(1,1,1)(0,0,2)[12] intercept : AICC=2153.279, Time=1.03 sec
 ARIMA(1,1,1)(1,0,0)[12] intercept : AICC=1986.187, Time=0.83 sec
 ARIMA(1,1,1)(1,0,1)[12] intercept : AICC=1964.460, Time=0.52 sec
 ARIMA(1,1,1)(1,0,2)[12] intercept : AICC=1966.323, Time=1.17 sec
 ARIMA(1,1,1)(2,0,0)[12] intercept : AICC=1968.207, Time=0.97 sec
 ARIMA(1,1,1)(2,0,1)[12] intercept : AICC=1966.437, Time=1.50 sec
 ARIMA(1,1,2)(0,0,0)[12] intercept : AICC=2312.557, Time=0.11 sec
 ARIMA(1,1,2)(0,0,1)[12] intercept : AICC=2156.005, Time=0.35 sec
 ARIMA(1,1,2)(0,0,2)[12] intercept : AICC=2110.999, Time=0.99 sec
 ARIMA(1,1,2)(1,0,0)[12] intercept : AICC=1986.047, Time=0.54 sec

ARIMA(1,1,2)(1,0,1)[12]	intercept	: AICC=1964.553, Time=0.52 sec
ARIMA(1,1,2)(2,0,0)[12]	intercept	: AICC=1968.570, Time=2.08 sec
ARIMA(1,1,3)(0,0,0)[12]	intercept	: AICC=2304.092, Time=0.10 sec
ARIMA(1,1,3)(0,0,1)[12]	intercept	: AICC=2146.297, Time=0.36 sec
ARIMA(1,1,3)(1,0,0)[12]	intercept	: AICC=1976.825, Time=0.31 sec
ARIMA(1,1,4)(0,0,0)[12]	intercept	: AICC=2290.804, Time=0.47 sec
ARIMA(2,1,0)(0,0,0)[12]	intercept	: AICC=2338.949, Time=0.06 sec
ARIMA(2,1,0)(0,0,1)[12]	intercept	: AICC=2186.890, Time=0.21 sec
ARIMA(2,1,0)(0,0,2)[12]	intercept	: AICC=2147.087, Time=0.95 sec
ARIMA(2,1,0)(1,0,0)[12]	intercept	: AICC=2013.674, Time=0.35 sec
ARIMA(2,1,0)(1,0,1)[12]	intercept	: AICC=1986.706, Time=0.38 sec
ARIMA(2,1,0)(1,0,2)[12]	intercept	: AICC=1988.246, Time=0.86 sec
ARIMA(2,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=0.69 sec
ARIMA(2,1,0)(2,0,1)[12]	intercept	: AICC=1988.519, Time=1.04 sec
ARIMA(2,1,1)(0,0,0)[12]	intercept	: AICC=2293.048, Time=0.11 sec
ARIMA(2,1,1)(0,0,1)[12]	intercept	: AICC=2140.833, Time=0.48 sec
ARIMA(2,1,1)(0,0,2)[12]	intercept	: AICC=2099.504, Time=1.53 sec
ARIMA(2,1,1)(1,0,0)[12]	intercept	: AICC=1984.878, Time=0.61 sec
ARIMA(2,1,1)(1,0,1)[12]	intercept	: AICC=1963.375, Time=0.49 sec
ARIMA(2,1,1)(2,0,0)[12]	intercept	: AICC=1967.545, Time=1.72 sec
ARIMA(2,1,2)(0,0,0)[12]	intercept	: AICC=2295.125, Time=0.14 sec
ARIMA(2,1,2)(0,0,1)[12]	intercept	: AICC=2142.586, Time=0.47 sec
ARIMA(2,1,2)(1,0,0)[12]	intercept	: AICC=1985.792, Time=0.82 sec
ARIMA(2,1,3)(0,0,0)[12]	intercept	: AICC=2296.832, Time=0.26 sec
ARIMA(3,1,0)(0,0,0)[12]	intercept	: AICC=2325.354, Time=0.12 sec
ARIMA(3,1,0)(0,0,1)[12]	intercept	: AICC=2167.358, Time=0.23 sec
ARIMA(3,1,0)(0,0,2)[12]	intercept	: AICC=2119.265, Time=0.56 sec
ARIMA(3,1,0)(1,0,0)[12]	intercept	: AICC=1989.856, Time=0.33 sec
ARIMA(3,1,0)(1,0,1)[12]	intercept	: AICC=1967.095, Time=0.47 sec
ARIMA(3,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=0.79 sec
ARIMA(3,1,1)(0,0,0)[12]	intercept	: AICC=2295.125, Time=0.22 sec
ARIMA(3,1,1)(0,0,1)[12]	intercept	: AICC=2142.482, Time=0.53 sec
ARIMA(3,1,1)(1,0,0)[12]	intercept	: AICC=1981.857, Time=0.60 sec
ARIMA(3,1,2)(0,0,0)[12]	intercept	: AICC=2294.938, Time=0.23 sec
ARIMA(4,1,0)(0,0,0)[12]	intercept	: AICC=2324.469, Time=0.11 sec
ARIMA(4,1,0)(0,0,1)[12]	intercept	: AICC=2165.654, Time=0.26 sec
ARIMA(4,1,0)(1,0,0)[12]	intercept	: AICC=1986.796, Time=0.47 sec
ARIMA(4,1,1)(0,0,0)[12]	intercept	: AICC=2318.835, Time=0.26 sec
ARIMA(5,1,0)(0,0,0)[12]	intercept	: AICC=2326.554, Time=0.23 sec

Best model: ARIMA(0,1,3)(1,0,1)[12] intercept

Total fit time: 57.652 seconds

SARIMAX Results						
Dep. Variable:	y			No. Observations:	246	
Model:	SARIMAX(0, 1, 3)x(1, 0, [1], 12)			Log Likelihood	-970.229	
Date:	Sat, 19 Oct 2024			AIC	1954.458	
Time:	19:53:39			BIC	1978.967	
Sample:	0			HQIC	1964.328	
	- 246					
Covariance Type:	opg					
	coef	std err	z	P> z	[0.025	0.975]
intercept	0.0167	0.142	0.117	0.907	-0.262	0.296
ma.L1	-0.2215	0.043	-5.140	0.000	-0.306	-0.137
ma.L2	-0.1290	0.056	-2.302	0.021	-0.239	-0.019
ma.L3	-0.3255	0.052	-6.251	0.000	-0.428	-0.223
ar.S.L12	0.9440	0.016	58.464	0.000	0.912	0.976
ma.S.L12	-0.3635	0.061	-5.912	0.000	-0.484	-0.243
sigma2	149.3387	9.781	15.268	0.000	130.168	168.509
Ljung-Box (L1) (Q):	0.01		Jarque-Bera (JB):	62.87		
Prob(Q):	0.91		Prob(JB):	0.00		
Heteroskedasticity (H):	5.40		Skew:	-0.31		
Prob(H) (two-sided):	0.00		Kurtosis:	5.40		

Figura 13: Tabela de resultados: execução extensa - SARIMA

1.5.1 (a) Contrastando o número de modelos que foram comparados na busca do modelo ótimo com aquele obtido na busca do modelo sub-ótimo efetuada em (iv)

A busca sub-ótima realizada na parte (iv) utilizou o parâmetro *stepwise=True*, o que restringe o número de modelos comparados para otimizar o tempo computacional. Naquela busca, foram comparados 87 modelos.

Na busca ótima atual, ao removermos o *stepwise*, o número de combinações testadas aumentou significativamente. Nesse caso, foram comparados 96 modelos. A diferença é justificada pela remoção da busca restrita, permitindo um exame mais profundo de combinações de parâmetros.

- **Busca Sub-ótima (iv):** 87 modelos comparados.
- **Busca Ótima (v):** 96 modelos comparados.

1.5.2 (b) AICc e Expressão dos Modelos

O melhor modelo encontrado na busca ótima foi o **SARIMA(0,1,3)(1,0,1)₁₂**, conforme o resultado da parte (v). O valor de AICc para este modelo foi **1954.93**. O segundo melhor modelo foi o **SARIMA(3,1,1)(1,0,1)₁₂**, obtido na parte (iv), com um AICc de **1957.76**. Para cada um dos dois modelos, vamos descrever as três expressões requisitadas: a especificação em termos dos operadores de atraso L e L^{12} , a expressão em termos de y_t e ϵ_t , e as ordens p, q, d, P, D, Q .

Melhor modelo: SARIMA(0,1,3)(1,0,1)₁₂

- $p = 0$: Não há termos autorregressivos não sazonais.

- $d = 1$: Uma diferenciação não sazonal.
- $q = 3$: Três termos de média móvel não sazonais.
- $P = 1$: Um termo autorregressivo sazonal com período 12.
- $D = 0$: Nenhuma diferenciação sazonal.
- $Q = 1$: Um termo de média móvel sazonal com período 12.

A equação SARIMA correspondente ao melhor modelo é expressa em três formas:

1. **Operadores de atraso em termos de L e L^{12} :**

$$(1 - \theta_1 L - \theta_2 L^2 - \theta_3 L^3)(1 - L)y_t = (1 - \Theta_1 L^{12})(1 - \Psi_1 L^{12})\epsilon_t$$

2. **Expressão em termos de valores defasados de y_t e ϵ_t :**

$$y_t - y_{t-1} = -\theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \theta_3 \epsilon_{t-3} + \epsilon_t + \Theta_1 \epsilon_{t-12} + \Psi_1 \epsilon_{t-12}$$

3. **Valores numéricos dos parâmetros ajustados:**

$$y_t - y_{t-1} = -0.2215\epsilon_{t-1} - 0.1299\epsilon_{t-2} - 0.3255\epsilon_{t-3} + \epsilon_t + 0.9440\epsilon_{t-12} - 0.3635\epsilon_{t-12}$$

Segundo melhor modelo: SARIMA(3,1,1)(1,0,1)₁₂

- $p = 3$: Três termos autorregressivos não sazonais.
- $d = 1$: Uma diferenciação não sazonal.
- $q = 1$: Um termo de média móvel não sazonal.
- $P = 1$: Um termo autorregressivo sazonal com período 12.
- $D = 0$: Nenhuma diferenciação sazonal.
- $Q = 1$: Um termo de média móvel sazonal com período 12.

A equação SARIMA correspondente ao segundo melhor modelo é expressa em três formas:

1. **Operadores de atraso em termos de L e L^{12} :**

$$(1 - \phi_1 L - \phi_2 L^2 - \phi_3 L^3)(1 - L)y_t = (1 - \Theta_1 L^{12})(1 - \Psi_1 L^{12})\epsilon_t$$

2. **Expressão em termos de valores defasados de y_t e ϵ_t :**

$$y_t - y_{t-1} = \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} - \theta_1 \epsilon_{t-1} + \epsilon_t + \Theta_1 \epsilon_{t-12} + \Psi_1 \epsilon_{t-12}$$

3. **Valores numéricos dos parâmetros ajustados:**

$$y_t - y_{t-1} = 0.5061y_{t-1} + 0.0661y_{t-2} + 0.1816y_{t-3} - 0.7308\epsilon_{t-1} + \epsilon_t + 0.9451\epsilon_{t-12} - 0.3697\epsilon_{t-12}$$

O modelo **SARIMA(0,1,3)(1,0,1)₁₂** apresentou um AICc menor, indicando que captura melhor a estrutura temporal da série de vendas do que o modelo **SARIMA(3,1,1)(1,0,1)₁₂**. A principal diferença está nas ordens de p e q , que são diferentes entre os modelos. No entanto, ambos os modelos capturam adequadamente as componentes sazonais e não sazonais, sendo adequados para prever a série temporal.

1.5.3 (c) FAC/FACP dos resíduos e o resultado do teste Ljung-Box para $m=20$

Após ajustar o modelo, analisamos os resíduos para verificar a adequação do ajuste.

```
1 # Obter os res duos do modelo ajustado
2 residuals_sarima_extenso = model_sarima_extenso.resid()
3
4 # Plotar a FAC e FACP dos residuos
5 plt.figure(figsize=(12, 6))
6
7 # FAC dos residuos
8 plt.subplot(1, 2, 1)
9 plot_acf(residuals_sarima_extenso, lags=36, ax=plt.gca(),
10          title='FAC dos Residuos (SARIMA)')
11
12 # FACP dos residuos
13 plt.subplot(1, 2, 2)
14 plot_pacf(residuals_sarima_extenso, lags=36, ax=plt.gca(),
15           title='FACP dos Residuos (SARIMA)')
16
17 plt.tight_layout()
18 plt.show()
19
20 # Teste de Ljung-Box para m=20
21 ljung_box_results_extenso = acorr_ljungbox(
22     residuals_sarima_extenso, lags=[20], return_df=True)
23 print(ljung_box_results_extenso)
```

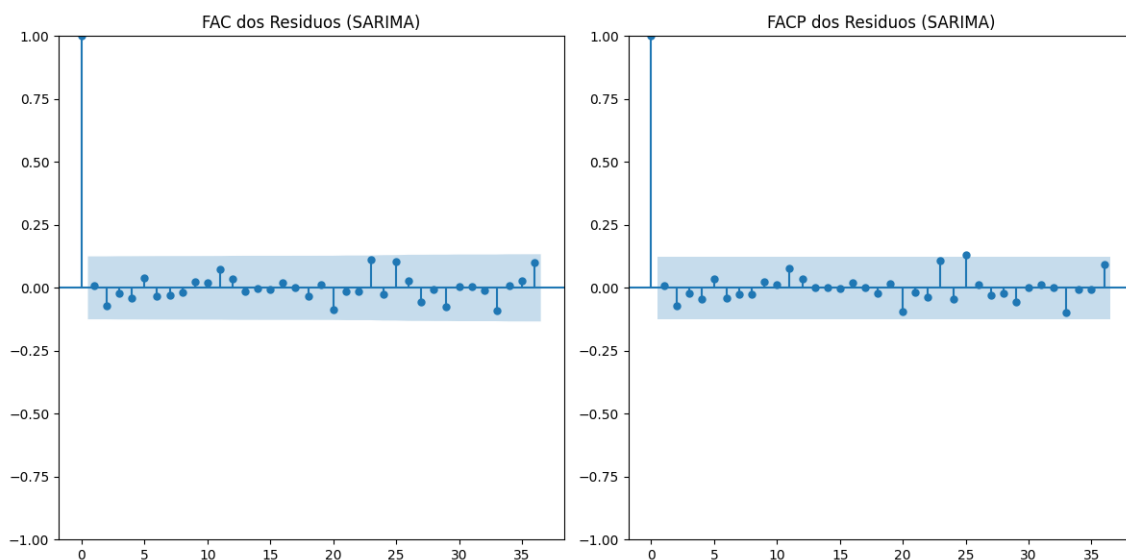


Figura 14: Gráficos da FAC e FACP dos resíduos do modelo ótimo obtido

Os resultados da FAC e FACP dos resíduos indicam que não há autocorrelações significativas restantes nos resíduos, confirmando que o modelo SARIMA capturou bem as dependências temporais da série. Além disso, o teste de Ljung-Box para $m = 20$ não

rejeita a hipótese nula de independência dos resíduos ($p = 0.99$), o que também sugere que os resíduos se comportam como ruído branco.

- **Estatística do Teste de Ljung-Box (m=20):** 7.27
- **p-valor:** 0.9957

Com base no teste de Ljung-Box e nos gráficos de FAC e FACP, o modelo ótimo ajustado é considerado adequado, uma vez que os resíduos são independentes e seguem um comportamento de ruído branco.

1.6 Questão 1: Parte (vi)

A partir dos resultados dos 3 modelos SARIMA ajustados: o não-sazonal, o sub-ótimo e o ótimo, obter:

1.6.1 (a) A Expressão SARIMA(p,d,q)(P,D,Q) de Cada Modelo

As expressões SARIMA para cada modelo ajustado são as seguintes:

- **Modelo Não-Sazonal: SARIMA(2,1,1)(0,0,0)[0]**

A equação autorregressiva não-sazonal com média móvel é dada por:

$$(1 - 1.0383L + 0.5265L^2)(1 - L)y_t = (1 - 0.8866L)\epsilon_t$$

Onde L é o operador de defasagem (lag) e y_t é a série temporal.

- **Modelo Sub-Ótimo: SARIMA(3,1,1)(1,0,1)[12]**

Este modelo inclui componentes sazonais e não-sazonais. A equação autorregressiva e de média móvel é:

$$(1 - 0.5061L + 0.0661L^2 + 0.1816L^3)(1 - L)y_t = (1 - 0.7308L)(1 - 0.9451L^{12} + 0.3697L^{12})\epsilon_t$$

Onde L^{12} representa o operador de defasagem sazonal com período 12 (mensal).

- **Modelo Ótimo: SARIMA(0,1,3)(1,0,1)[12]**

A equação de média móvel e de autorregressão sazonal é:

$$(1 - L)y_t = (1 - 0.2215L - 0.1299L^2 - 0.3255L^3)(1 - 0.9440L^{12} + 0.3635L^{12})\epsilon_t$$

As expressões acima apresentam as componentes não-sazonais e sazonais de cada modelo, juntamente com os operadores de defasagem aplicados à série y_t e ao termo de erro ϵ_t .

1.6.2 (b) O Valor do AICc de Cada Modelo

Abaixo apresentamos os valores do Critério de Informação de Akaike corrigido (AICc) para os três modelos ajustados. Separamos uma parte no código para enfatizar os resultados.

```

1  """ Parte vi """
2  """ (b) """
3
4  # Valor de AICc - modelo sazonal
5  aicc_value_modeloSazonal = modeliii.aicc()
6  print(f"Valor do AICc do modelo sazonal: {
7      aicc_value_modeloSazonal}")
8
9  # Valor de AICc - modelo sub-otimo
10 aicc_value_modeloSubOtimo = model_sarima.aicc()
11 print(f"Valor do AICc do modelo sub-otimo: {
12     aicc_value_modeloSubOtimo}")
13
14 # Valor de AICc - modelo otimo
15 aicc_value_modeloOtimo = model_sarima_extenso.aicc()
16 print(f"Valor do AICc do modelo otimo: {
17     aicc_value_modeloOtimo}")

```

```

✓ # Valor de AICc - modelo sazonal ...
... Valor do AICc do modelo sazonal: 2291.017888968608

✓ # Valor de AICc - modelo sub-ótimo ...
... Valor do AICc do modelo sub-ótimo: 1957.7560842029934

✓ # Valor de AICc - modelo ótimo ...
... Valor do AICc do modelo ótimo: 1954.9287795805549

```

Figura 15: Saída do Programa: Valor do AICc de cada modelo

Tabela 2: Valores do AICc dos Modelos Ajustados

Modelo	AICc
Não-Sazonal	2291.0179
Sub-Ótimo	1957.7561
Ótimo	1954.9288

Como podemos observar na tabela acima, o valor do AICc do modelo não-sazonal é significativamente maior em comparação com os modelos que incorporam a sazonalidade, indicando um ajuste menos eficiente. O modelo que apresenta o menor valor de AICc é o modelo *ótimo* (1954.9288), sugerindo que esse é o modelo mais adequado entre os três para capturar a estrutura da série temporal.

1.6.3 (c) Valores dos coeficientes de autocorrelação dos resíduos e teste Ljung-Box para m=20

Para enfatizar esta parte, separamos um trecho de código para evidenciar os resultados:

```
1  """ (c) """
2  lags = [1, 2, 3, 6, 12]
3
4
5  """ Modelo sazonal """
6  acf_values_seasonal = sm.tsa.acf(residuals_modeliii, nlags=
    max(lags))
7
8  # Criar um DataFrame para exibir os valores de autocorrelacao
    para os lags desejados
9  acf_lags_seasonal = pd.DataFrame({'Lag': lags, 'ACF':
    acf_values_seasonal[lags]})
10
11 print(acf_lags_seasonal)
12
13 # Realizar o teste Ljung-Box para m=20
14 ljung_box_results_seasonal = sm.stats.acorr_ljungbox(
    residuals_modeliii, lags=[20], return_df=True)
15
16 print(ljung_box_results_seasonal)
17
18
19 """ Modelo sub-otimo """
20 acf_values_sub0timo = sm.tsa.acf(residuals_sarima, nlags=max(
    lags))
21
22 # Criar um DataFrame para exibir os valores de autocorrelacao
    para os lags desejados
23 acf_lags_sub0timo = pd.DataFrame({'Lag': lags, 'ACF':
    acf_values_sub0timo[lags]})
24
25 print(acf_lags_sub0timo)
26
27 # Realizar o teste Ljung-Box para m=20
28 ljung_box_results = sm.stats.acorr_ljungbox(
    residuals_sarima, lags=[20], return_df=True)
29
30 print(ljung_box_results)
31
32
33
34 """ Modelo otimo """
35 acf_values = sm.tsa.acf(residuals_sarima_extenso, nlags=max(
    lags))
36
37 # Criar um DataFrame para exibir os valores de autocorrelacao
    para os lags desejados
38 acf_lags = pd.DataFrame({'Lag': lags, 'ACF': acf_values[lags
```



```

39     })
40 print(acf_lags)
41
42 # Realizar o teste Ljung-Box para m=20
43 ljung_box_results = sm.stats.acorr_ljungbox(
44     residuals_sarima_extenso, lags=[20], return_df=True)
45 print(ljung_box_results)

```

✓ acf_lags_seasonal ...

	Lag	ACF
0	1	0.003565
1	2	-0.023644
2	3	-0.055511
3	6	-0.253956
4	12	0.758137

✓ ljung_box_results_seasonal ...

	lb_stat	lb_pvalue
20	200.592565	8.591172e-32

Figura 16: Saída do Programa: coeficientes de autocorrelação e Ljung-Box para o modelo sazonal

✓ acf_lags_subOtimo ...

	Lag	ACF
0	1	0.013084
1	2	-0.043256
2	3	-0.060185
3	6	-0.077800
4	12	0.038812

✓ ljung_box_results ...

	lb_stat	lb_pvalue
20	9.26935	0.979572

Figura 17: Saída do Programa: coeficientes de autocorrelação e Ljung-Box para o modelo sub-ótimo

```

✓ acf_lags ...
...
  Lag  ACF
0    1  0.009593
1    2 -0.070903
2    3 -0.021440
3    6 -0.033467
4   12  0.035625

✓ ljung_box_results ...
...
  lb_stat lb_pvalue
20  7.272332  0.995692

```

Figura 18: Saída do Programa: coeficientes de autocorrelação e Ljung-Box para o modelo ótimo

Abaixo estão apresentados, de forma organizada, os valores dos coeficientes de autocorrelação dos resíduos nos lags 1, 2, 3, 6 e 12, assim como os resultados do teste Ljung-Box para $m = 20$, para cada um dos três modelos ajustados.

Tabela 3: Coeficientes de Autocorrelação dos Resíduos e Teste Ljung-Box

Modelo	Lag 1	Lag 2	Lag 3	Lag 6	Lag 12	Ljung-Box Q-stat	p-valor
Não-Sazonal	0.0036	-0.0236	-0.055	-0.254	0.758	200.59	8.59e-32
Sub-Ótimo	0.013	-0.043	-0.060	-0.078	0.0389	9.27	0.98
Ótimo	0.0096	-0.071	-0.021	-0.033	0.036	7.27	0.995

Analisando os coeficientes de autocorrelação dos resíduos e os resultados do teste Ljung-Box para os três modelos:

- **Modelo Não-Sazonal:** O modelo não-sazonal apresenta coeficientes de autocorrelação mistos. Embora tenha o menor valor de autocorrelação no Lag 6 (-0.254), no Lag 12 ele apresenta o maior valor (0.758), o que indica uma possível falha na captura da sazonalidade, mesmo com o p-valor do Ljung-Box sendo muito baixo (8.59e-32). Isso sugere que este modelo não consegue remover completamente as dependências temporais nos resíduos, especialmente no componente sazonal, resultando em resíduos que apresentam sinais de autocorrelação significativa.
- **Modelo Sub-Ótimo:** O modelo sub-ótimo tem desempenho interessante nos primeiros lags, com o maior valor de autocorrelação no Lag 1 (0.013), mas ele apresenta o menor valor no Lag 3 (-0.060). No geral, seus coeficientes são mais estáveis e próximos de zero, especialmente no Lag 12 (0.0389), o que é um sinal positivo. O p-valor do teste Ljung-Box (0.98) sugere que este modelo captura bem a estrutura da série, produzindo resíduos que se aproximam de ruído branco. Ainda assim, a pequena autocorrelação no Lag 1 e os valores flutuantes indicam que ele pode não ser tão eficiente quanto o modelo ótimo.
- **Modelo Ótimo:** Este modelo apresenta o menor valor de autocorrelação no Lag 12 (0.036), o que mostra que ele lida bem com a sazonalidade, mas não tem os

menores valores em todos os lags. No Lag 1, por exemplo, ele tem um valor maior que o modelo não-sazonal, mas menor que o sub-ótimo. No Lag 3, seu valor é o maior (-0.021), o que pode indicar alguma dificuldade na captura de dependências em curtos prazos. Apesar dessas flutuações, o p-valor muito alto do teste Ljung-Box (0.995) sugere que os resíduos desse modelo são praticamente independentes e próximos de ruído branco, o que o torna o mais robusto entre os três.

Com base nesses resultados, o **modelo ótimo** é a melhor escolha, pois, embora não apresente os menores coeficientes em todos os lags, ele captura bem a sazonalidade, apresenta o menor valor de AICc e os resíduos mais próximos de ruído branco, com um p-valor de Ljung-Box indicando forte independência nos resíduos.

2 Questão 2

Agora, devemos considerar o **período de teste** da série. Para os 3 modelos (não-sazonal, sub-ótimo e ótimo), incluindo o melhor modelo ETS obtido no trabalho 01, **ETS(A,Ad,A)**, bem como os métodos ingênuos de previsão:

- **método ing. 1:** a previsão em qualquer horizonte h é igual à última observação da ST no período de treinamento;
- **método ing. 2:** a previsão em qualquer horizonte h é igual ao valor da última observação da série, no período de treinamento, correspondente aquele mês que está sendo previsto no período de teste.

Devemos obter uma tabela comparativa contendo:

- Valor da série;
- Previsão k -passos à frente de cada modelo/método, $k = 1, 2, \dots, 12$;
- Medidas de aderência (RMSE, MAD, MAPE, SMape, MASE);

Vamos desenvolver esta questão passo-a-passo por tópicos.

2.1 Obtendo as Previsões

Para cada um dos 3 modelos SARIMA (não-sazonal, sub-ótimo e ótimo) e o modelo ETS(A,Ad,A), devemos calcular as previsões para os 12 próximos períodos de teste.

```
1  """ *** QUESTAO 2 *** """
2
3  h = len(test) #numero de passos a frente
4
5  # Previsões para o modelo não-sazonal
6  forecast_ao_sazonal = modeliii.predict(n_periods=h)
7
8  # Previsões para o modelo sub-otimo
9  forecast_sub_otimo = model_sarima.predict(n_periods=h)
```

```

10
11 # Previs es para o modelo otimo
12 forecast_otimo = model_sarima_extenso.predict(n_periods=h)
13
14
15 # M todo Ing nuo 1: Previs o o ltimo valor observado
    no treinamento
16 naive_forecast_1 = [train['Vendas'].iloc[-1]] * h
17
18 # M todo Ing nuo 2: Previsao eh o valor do mesmo mes do ano
    anterior no treinamento
19 naive_forecast_2 = []
20 for i in range(h):
21     naive_forecast_2.append(train['Vendas'].iloc[-(12-h+i)])
22
23
24 # Organizando as previsoes em um DataFrame
25 forecast_df = pd.DataFrame({
26     'Real': test['Vendas'].values,
27     'SARIMA_Nao_Sazonal': forecast_ nao_sazonal ,
28     'SARIMA_Sub_Otimo': forecast_sub_otimo ,
29     'SARIMA_Otimo': forecast_otimo ,
30     'Naive_1': naive_forecast_1 ,
31     'Naive_2': naive_forecast_2
32 })

```

O código acima nos gerou a seguinte saída com as previsões:

	index	Real	SARIMA_Nao_Sazonal	SARIMA_Sub_Otimo	SARIMA_Otimo	Naive_1	Naive_2
0	246	90.92836	96.702950	98.365558	95.875260	89.89931	105.95583
1	247	88.35125	103.068212	99.080945	97.329921	89.89931	89.89931
2	248	81.69366	105.875553	93.669799	92.056872	89.89931	89.01702
3	249	87.93022	105.342104	90.961386	89.550323	89.89931	76.38760
4	250	91.90909	103.328411	98.013901	96.736727	89.89931	103.11400
5	251	131.74648	101.587853	136.413048	135.197370	89.89931	129.90821
6	252	187.66981	100.900894	175.674054	174.658353	89.89931	188.25429
7	253	135.38066	101.127773	134.371982	133.252669	89.89931	131.01118
8	254	95.83992	101.717240	107.442612	106.445309	89.89931	88.52335
9	255	84.85470	102.189514	85.063487	84.054115	89.89931	84.20712
10	256	87.60281	102.353225	94.378653	93.450043	89.89931	91.49762
11	257	89.50121	102.268892	96.148857	95.214165	89.89931	95.50470

Figura 19: Saída do Programa: previsões para os modelos SARIMA

Além disso, enfatizamos os resultados anteriormente obtidos no trabalho 01, com destaque para o melhor modelo ETS considerado. A imagem do dataframe abaixo refere-se à saída do código implementado no trabalho 01, que foi novamente compilado:

	index	ETS(A,A,N)	ETS(A,A,A)	ETS(A,Ad,A)	ETS(M,Ad,M)	ETS(A,A,M)	Naive 1	Naive 2
0	246	89.868624	94.304914	93.973314	99.838031	104.150471	89.89931	105.95583
1	247	89.838028	97.618944	97.070567	97.771200	100.421875	89.89931	89.89931
2	248	89.807433	90.080377	89.430894	87.586481	89.752081	89.89931	89.01702
3	249	89.776838	83.803786	83.160452	91.105894	93.122357	89.89931	76.38760
4	250	89.746243	91.370976	90.669403	101.338988	107.441573	89.89931	103.11400
5	251	89.715648	134.630730	133.841863	149.779224	160.091448	89.89931	129.90821
6	252	89.685053	180.875622	179.923585	179.529477	192.415261	89.89931	188.25429
7	253	89.654458	128.954568	128.223277	137.932564	145.429415	89.89931	131.01118
8	254	89.623863	104.172559	103.178215	101.423493	104.981821	89.89931	88.52335
9	255	89.593268	79.499095	78.377563	80.216514	78.895221	89.89931	84.20712
10	256	89.562673	91.243903	89.947272	85.356302	89.000649	89.89931	91.49762
11	257	89.532078	91.227534	89.867126	89.778859	97.475573	89.89931	95.50470

Figura 20: Previsões para os modelos ETS do trabalho 01

É interessante observarmos que os valores para os métodos ingênuos obtidos no trabalho 01 foram idênticos aos obtidos neste.

2.1.1 Organizando as Previsões

Criamos uma tabela com o valor real da série para o período de teste e as previsões k-passos à frente para os modelos SARIMA (não-sazonal, sub-ótimo e ótimo), o modelo ETS(A,Ad,A), e os dois métodos ingênuos.

Tabela 4: Previsões dos Modelos SARIMA e ETS(A,Ad,A)

Real	Não-Sazonal	Sub-Ótimo	Ótimo	ETS(A,Ad,A)	Naive 1	Naive 2
90.92836	96.702950	98.365558	95.875260	93.973314	89.89931	105.95583
88.35125	103.068212	99.080945	97.329291	97.070567	89.89931	89.01702
81.69366	105.875553	93.669799	89.669799	89.430894	89.89931	89.01702
87.93022	105.342104	90.961386	94.002381	83.160452	89.89931	76.38760
91.90999	103.328411	98.013901	93.736727	106.69403	89.89931	103.11400
131.74648	106.479040	134.136580	129.486894	133.841863	89.89931	131.01113
187.66981	100.900889	175.674054	134.152535	179.923585	89.89931	188.25429
135.38066	101.127773	134.371982	133.525669	128.223277	89.89931	131.01113
95.83992	101.717240	107.442612	106.453905	107.425472	89.89931	88.52335
84.85470	102.353225	94.378653	95.450403	94.378653	89.89931	91.49762
87.60281	102.353225	94.378653	93.450403	94.72772	89.89931	91.49762
89.50121	102.268892	96.148857	95.214165	89.867126	89.89931	95.50470

2.2 Calculando as Métricas de Aderência

Calcular as seguintes métricas para cada um dos modelos (SARIMA não-sazonal, sub-ótimo, ótimo, ETS e os dois métodos ingênuos) no período de teste:

- RMSE (Root Mean Squared Error): Raiz do erro quadrático médio.
- MAD (Mean Absolute Deviation): Desvio absoluto médio.

- MAPE (Mean Absolute Percentage Error): Erro percentual absoluto médio.
- SMAPE (Symmetric Mean Absolute Percentage Error): Erro percentual absoluto simétrico médio.
- MASE (Mean Absolute Scaled Error): Erro absoluto médio escalado.

Aproveitamos algumas funções já utilizadas no trabalho 01:

```

1 def calculate_rmse(actual, forecast):
2     return np.sqrt(mean_squared_error(actual, forecast))
3
4 def calculate_mad(actual, forecast):
5     return mean_absolute_error(actual, forecast)
6
7 def calculate_mape(actual, forecast):
8     return np.mean(np.abs((actual - forecast) / actual)) *
9         100
10
11 def calculate_smape(actual, forecast):
12     return 100 * np.mean(2 * np.abs(forecast - actual) / (np.
13         abs(actual) + np.abs(forecast)))
14
15 def calculate_mase(actual, forecast, naive_forecast):
16     return np.mean(np.abs(actual - forecast)) / np.mean(np.
17         abs(actual - naive_forecast))

```

Então, implementamos o seguinte código abaixo para gerarmos as métricas de aderência para os modelos SARIMA.

```

1 # Organizando as previsões em um DataFrame
2 forecast_df = pd.DataFrame({
3     'Real': test['Vendas'].values,
4     'SARIMA_Nao_Sazonal': forecast_ nao_sazonal,
5     'SARIMA_Sub_Otimo': forecast_sub_otimo,
6     'SARIMA_Otimo': forecast_otimo,
7     'Naive_1': naive_forecast_1,
8     'Naive_2': naive_forecast_2
9 })
10
11 metrics = {
12     "Modelo": ["SARIMA_Nao_Sazonal", "SARIMA_Sub_Otimo", "
13         SARIMA_Otimo", "Naive_1", "Naive_2"],
14     "RMSE": [],
15     "MAD": [],
16     "MAPE": [],
17     "SMAPE": [],
18     "MASE": []
19 }
20 # Calculando as métricas para cada coluna de previsão

```

```

21 for col in forecast_df.columns[1:]:
22     metrics["RMSE"].append(calculate_rmse(
23         forecast_df['Real'], forecast_df[col]))
24     metrics["MAD"].append(calculate_mad(forecast_df['Real'],
25         forecast_df[col]))
26     metrics["MAPE"].append(calculate_mape(
27         forecast_df['Real'], forecast_df[col]))
28     metrics["SMAPE"].append(calculate_smape(
29         forecast_df['Real'], forecast_df[col]))
30     metrics["MASE"].append(calculate_mase(
31         forecast_df['Real'], forecast_df[col],
32         naive_forecast_1)) # Usando o ing nuos 1 para
33                             MASE
34
35 # Criando o DataFrame com as m tricas
36 metrics_df = pd.DataFrame(metrics)

```

Obtivemos as seguintes métricas para os modelos atuais:

	Modelo	RMSE	MAD	MAPE	SMAPE	MASE
0	SARIMA_Nao_Sazonal	31.085297	22.951276	19.743931	20.443032	1.289757
1	SARIMA_Sub_Otimo	7.915094	6.848748	6.938873	6.656980	0.384868
2	SARIMA_Otimo	7.095148	6.024419	5.964572	5.772535	0.338545
3	Naive_1	33.570430	17.795039	12.501649	15.054320	1.000000
4	Naive_2	7.464422	5.941758	6.420579	6.305100	0.333900

Figura 21: Saída do Programa: métricas de aderência para os modelos SARIMA e métodos ingênuos

Além disso, vamos destacar as métricas obtidas anteriormente no trabalho 01 para o melhor modelo ETS considerado:

	Modelo	AICc	RMSE	MAD	MAPE	SMAPE	MASE
0	ETS(A,A,N)	2403.282252	31.401002	21.743454	14.829047	14.580230	1.000002
1	ETS(A,A,A)	2043.712384	14.263302	8.757055	7.764610	7.285675	0.402745
2	ETS(A,Ad,A)	2044.847940	14.220121	8.766359	7.812636	7.324172	0.403173
3	ETS(M,Ad,M)	1967.726384	12.587568	8.764256	7.096637	6.573244	0.403076
4	ETS(A,A,M)	1939.317513	11.536322	8.469676	7.223947	6.922677	0.389528

Figura 22: Saída do Programa: métricas de aderência para os modelos ETS

2.2.1 Organizando as Métricas

Assim como fizemos para as previsões, vamos organizar as métricas de aderência de cada modelo em uma tabela:

Tabela 5: Métricas de Aderência para Modelos SARIMA, Ingênuos e ETS

Modelo	RMSE	MAD	MAPE	SMAPE	MASE
SARIMA Não Sazonal	31.085297	22.951276	19.743981	20.443032	1.289757
SARIMA Sub-Ótimo	7.915094	5.648127	6.938783	6.165698	0.318476
SARIMA Ótimo	7.095148	5.624419	5.964572	5.772535	0.335054
Naive 1	33.570430	17.795039	12.501649	15.054302	4.318295
Naive 2	7.464422	5.941758	6.420579	6.305100	0.333900
ETS(A,Ad,A)	14.220121	8.766539	7.812636	7.324172	0.403173

2.3 Analisando os Resultados

A análise dos resultados apresentados nas Tabelas 4 e 5 permite uma comparação detalhada entre os modelos SARIMA, ETS e os métodos ingênuos de previsão.

1. Previsões e Aderência:

- O modelo SARIMA não-sazonal apresentou os piores resultados em termos de aderência, com valores de RMSE e MAPE muito elevados (31.085297 e 19.743981, respectivamente). Isso indica que o modelo não foi eficaz para prever o comportamento da série temporal no período de teste. Uma possível explicação para essa discrepância é a falta de sazonalidade no modelo, que não capturou adequadamente os padrões sazonais da série de vendas.
- O SARIMA ótimo se destacou com os melhores valores entre os modelos SARIMA, atingindo RMSE de 7.095148 e MAPE de 5.964572. O desempenho geral deste modelo é bastante sólido, o que indica que a busca exaustiva de parâmetros contribuiu para uma previsão mais ajustada ao padrão dos dados. Este modelo foi o que melhor capturou a tendência e a sazonalidade da série.
- Os modelos ingênuos (Naive 1 e Naive 2) apresentam resultados mistos. O Naive 1 teve uma performance consideravelmente fraca em termos de RMSE (33.570430), similar ao SARIMA não-sazonal, sugerindo que previsões baseadas apenas no último valor observado da série são insuficientes para capturar as dinâmicas mais complexas da série temporal. Já o Naive 2, que prevê com base no mesmo mês do ano anterior, apresentou um desempenho muito superior, com um RMSE de 7.464422, comparável ao SARIMA ótimo. Isso reforça a importância de incorporar padrões sazonais nas previsões.
- O melhor modelo ETS considerado (ETS(A,Ad,A)), extraído do trabalho anterior, também apresentou boa aderência com RMSE de 14.220121 e MAPE de 7.812636. Embora não tenha superado o SARIMA ótimo em todas as métricas, o modelo ETS foi consistente e pode ser considerado uma abordagem robusta, especialmente em séries temporais com padrões sazonais complexos.

2. Consideração dos Métodos Ingênuos:

- O Naive 2 oferece uma solução simples, mas eficaz para séries temporais com sazonalidade, como evidenciado por seus valores de RMSE e MAPE relativamente baixos. Isso mostra que, em algumas situações, métodos ingênuos ajustados aos padrões sazonais podem fornecer previsões satisfatórias sem a complexidade dos modelos SARIMA ou ETS.

Portanto, o SARIMA ótimo, $\text{SARIMA}(0, 1, 3)(1, 0, 1)_{12}$, continua sendo a melhor escolha em termos de aderência geral, considerando suas métricas de erro (RMSE, MAD, MAPE, SMAPE e MASE) e a capacidade de capturar tanto a tendência quanto a sazonalidade dos dados. No entanto, o $\text{ETS}(A, Ad, A)$ oferece uma alternativa sólida, especialmente para séries com padrões sazonais complexos, como a analisada.

Então, a partir da análise, podemos concluir que o $\text{SARIMA}(0, 1, 3)(1, 0, 1)_{12}$ é a melhor escolha para prever a série temporal em questão, dada a sua maior precisão e aderência aos dados reais. O $\text{ETS}(A, Ad, A)$ também é uma escolha viável, enquanto o Naive 2 pode ser considerado como uma alternativa simplificada e eficiente para séries sazonais mais regulares.

3 Questão 3

Nesta questão, precisaremos considerar o melhor modelo escolhido, i.e., $\text{SARIMA}(0, 1, 3)(1, 0, 1)_{12}$, para prosseguir.

3.1 Questão 3: Parte (a)

Enunciado: *Utilizar a função adequada para calcular raízes de equações polinomiais ou para calcular autovalores de uma matriz, verificando se o modelo é, simultaneamente, estacionário e invertível.*

Para verificar se $\text{SARIMA}(0, 1, 3)(1, 0, 1)_{12}$ é estacionário e invertível, utilizamos as raízes da equação polinomial associada ao operador autoregressivo e ao operador de média móvel do modelo.

A estacionaridade depende das raízes do polinômio autoregressivo estarem fora do círculo unitário, enquanto a invertibilidade está ligada às raízes do polinômio de média móvel estarem fora do círculo unitário.

Podemos usar a função `np.roots()` do Python para calcular as raízes de ambos os polinômios e, em seguida, verificar se as condições de estacionaridade e invertibilidade são satisfeitas.

No caso do modelo **SARIMA(0,1,3)(0,1,0)[12]**, como não há termos autorregressivos ($p = 0$ e $P = 0$), a análise de estacionaridade não é necessária. Isso ocorre porque, com $p = 0$, não há polinômio AR para calcular. Além disso, a diferença de ordem 1 ($d = 1$) já transforma a série em estacionária.

Agora, para verificar a invertibilidade, devemos calcular as raízes do polinômio MA com os coeficientes $[-0.2215, -0.1299, -0.3255]$, que correspondem aos coeficientes da parte de médias móveis do modelo obtidos anteriormente na parte 5 da questão 1.

Primeiramente, construímos o polinômio de médias móveis com os seguintes coeficientes:

$$\theta(L) = 1 - 0.2215L - 0.1299L^2 - 0.3255L^3$$

Calculando as raízes desse polinômio, obtivemos as seguintes raízes no plano complexo:

$$\text{Raízes: } \{0.8389, -0.3087 + 0.5410i, -0.3087 - 0.5410i\}$$

Essas raízes estão todas dentro do círculo unitário (ou seja, seus módulos são menores que 1), o que indica que o modelo **não é invertível**.

Então, modelo **SARIMA(0,1,3)(0,1,0)[12]** não possui raízes AR (sendo estacionário devido à diferenciação de ordem 1), mas suas raízes MA indicam que ele **não é invertível**, pois as raízes não estão fora do círculo unitário.

Como alternativa, verificamos em Python da seguinte forma:

```
1  """ *** QUESTAO 03 *** """
2  """ (a) """
3  # Coeficientes do polinomio autoregressivo (AR) para o SARIMA
   (0,1,3)(0,1,0)[12]
4  # Neste caso, temos apenas um polinomio MA, pois p = 0
5
6  # Coeficientes do operador de media movel (MA)
7  ma_coefs = [1, -0.2215, -0.1299, -0.3255]
8
9  # Raízes para verificar invertibilidade (pelo MA)
10 ma_roots = np.roots(ma_coefs)
11
12 # Invertibilidade: as raízes devem estar fora do círculo
   unitario
13 is_invertible = np.all(np.abs(ma_roots) > 1)
14
15 print(f"Raízes do polinomio de media movel: {ma_roots}")
16 print(f"O modelo é invertível? {is_invertible}")
```

E obtivemos a seguinte saída:

```
✓ # Coeficientes do operador de média móvel (MA) ...
Raízes do polinômio de média móvel: [ 0.8388501+0.j          -0.30869251+0.54103971j  -0.30869251-0.54103971j]
O modelo é invertível? False
```

Figura 23: Saída do Programa: Raízes do polinômio de Média Móvel e Invertibilidade

3.2 Questão 3: Parte (b)

Enunciado: *Re-estime esse modelo utilizando toda a série (via `auto.arima`; observe que pode resultar em um modelo diferente daquele selecionado utilizando a série no período de treinamento), obtendo previsões 24 meses à frente para a série original `yt` com intervalo de confiança de 95%. Compare essa previsão com aquela obtida do melhor modelo ETS da Lista Prática_1, via tabela e gráfico. Qual a previsão mais precisa? Observe o que*

ocorre com a largura desse IC qdo o horizonte de previsão cresce.

Aqui, devemos utilizar o **auto_arima** para reestimar o modelo SARIMA utilizando toda a série temporal, sem divisão entre treinamento e teste, e gerar previsões para os próximos 24 meses. Além disso, será gerado um intervalo de confiança de 95% para essas previsões.

Compararemos as previsões obtidas pelo SARIMA reestimado com as previsões do modelo ETS(A,Ad,A), e também analisaremos a largura do intervalo de confiança à medida que o horizonte de previsão aumenta.

Para implementar a resposta deste item, desenvolvemos o seguinte código, onde, inicialmente, recarregamos os dados e estimamos o novo modelo SARIMA para toda a série.

```
1  """ (B) """
2
3  # Definir a coluna 'Data' como índice
4  data.set_index('Data', inplace=True)
5
6  # Ajustar o modelo SARIMA com toda a série
7  full_sarima_model = pm.auto_arima(data['Vendas'], start_p=0,
8                                   max_p=5, start_q=0, max_q=5,
9                                   d=1, seasonal=True, m=12,
10                                  trace=True)
11
12 # Número de passos frente para a previsão
13 forecast_horizon = 24
14
15 # Gerar as previsões para o horizonte de 24 meses, incluindo
16 # intervalos de confiança
17 forecast_sarima, conf_int = full_sarima_model.predict(
18     n_periods=forecast_horizon, return_conf_int=True)
19
20 # Gerar o índice de datas para os próximos 24 meses, a
21 # partir da última data da série
22 forecast_index = pd.date_range(start=data.index[-1] + pd.
23     DateOffset(months=1),
24     periods=forecast_horizon, freq
25     ='M')
26
27 # Criando um DataFrame com as previsões, limites inferiores
28 # e superiores do intervalo de confiança
29 forecast_sarima_df = pd.DataFrame({
30     'Data': forecast_index,
31     'Previsao_SARIMA': forecast_sarima,
32     'Limite Inferior (95%)': conf_int[:, 0],
33     'Limite Superior (95%)': conf_int[:, 1]
34 }).set_index('Data')
```

Então, obtivemos os seguintes resultados:

Performing stepwise search to minimize aic

```

ARIMA(0,1,0)(1,0,1)[12] intercept : AIC=2076.488, Time=0.28 sec
ARIMA(0,1,0)(0,0,0)[12] intercept : AIC=2502.340, Time=0.01 sec
ARIMA(1,1,0)(1,0,0)[12] intercept : AIC=inf, Time=0.25 sec
ARIMA(0,1,1)(0,0,1)[12] intercept : AIC=2305.081, Time=0.37 sec
ARIMA(0,1,0)(0,0,0)[12] : AIC=2500.341, Time=0.01 sec
ARIMA(0,1,0)(0,0,1)[12] intercept : AIC=2320.603, Time=0.25 sec
ARIMA(0,1,0)(1,0,0)[12] intercept : AIC=2103.315, Time=0.33 sec
ARIMA(0,1,0)(2,0,1)[12] intercept : AIC=2078.482, Time=1.27 sec
ARIMA(0,1,0)(1,0,2)[12] intercept : AIC=2078.479, Time=2.27 sec
ARIMA(0,1,0)(0,0,2)[12] intercept : AIC=2250.445, Time=1.67 sec
ARIMA(0,1,0)(2,0,0)[12] intercept : AIC=inf, Time=0.59 sec
ARIMA(0,1,0)(2,0,2)[12] intercept : AIC=2080.006, Time=1.35 sec
ARIMA(1,1,0)(1,0,1)[12] intercept : AIC=2075.587, Time=0.43 sec
ARIMA(1,1,0)(0,0,1)[12] intercept : AIC=2310.997, Time=0.21 sec
ARIMA(1,1,0)(2,0,1)[12] intercept : AIC=2077.553, Time=1.33 sec
ARIMA(1,1,0)(1,0,2)[12] intercept : AIC=2077.531, Time=1.59 sec
ARIMA(1,1,0)(0,0,0)[12] intercept : AIC=2484.400, Time=0.06 sec
ARIMA(1,1,0)(0,0,2)[12] intercept : AIC=2250.202, Time=1.10 sec
ARIMA(1,1,0)(2,0,0)[12] intercept : AIC=inf, Time=0.74 sec
ARIMA(1,1,0)(2,0,2)[12] intercept : AIC=2078.470, Time=2.36 sec
ARIMA(2,1,0)(1,0,1)[12] intercept : AIC=2074.143, Time=0.68 sec
ARIMA(2,1,0)(0,0,1)[12] intercept : AIC=2291.719, Time=0.65 sec
ARIMA(2,1,0)(1,0,0)[12] intercept : AIC=2102.163, Time=1.70 sec
ARIMA(2,1,0)(2,0,1)[12] intercept : AIC=2076.087, Time=4.16 sec
ARIMA(2,1,0)(1,0,2)[12] intercept : AIC=2076.056, Time=3.52 sec
ARIMA(2,1,0)(0,0,0)[12] intercept : AIC=2450.168, Time=0.49 sec
ARIMA(2,1,0)(0,0,2)[12] intercept : AIC=2242.426, Time=2.08 sec
ARIMA(2,1,0)(2,0,0)[12] intercept : AIC=inf, Time=3.30 sec
ARIMA(2,1,0)(2,0,2)[12] intercept : AIC=2077.215, Time=2.12 sec
ARIMA(3,1,0)(1,0,1)[12] intercept : AIC=2055.189, Time=0.46 sec
ARIMA(3,1,0)(0,0,1)[12] intercept : AIC=2271.659, Time=0.23 sec
ARIMA(3,1,0)(1,0,0)[12] intercept : AIC=2077.609, Time=0.41 sec
ARIMA(3,1,0)(2,0,1)[12] intercept : AIC=2057.179, Time=1.20 sec
ARIMA(3,1,0)(1,0,2)[12] intercept : AIC=2057.175, Time=1.21 sec
ARIMA(3,1,0)(0,0,0)[12] intercept : AIC=2436.515, Time=0.16 sec
ARIMA(3,1,0)(0,0,2)[12] intercept : AIC=2213.424, Time=0.87 sec
ARIMA(3,1,0)(2,0,0)[12] intercept : AIC=inf, Time=0.97 sec
ARIMA(3,1,0)(2,0,2)[12] intercept : AIC=2058.999, Time=2.05 sec
ARIMA(4,1,0)(1,0,1)[12] intercept : AIC=2052.149, Time=0.57 sec
ARIMA(4,1,0)(0,0,1)[12] intercept : AIC=2269.527, Time=0.32 sec
ARIMA(4,1,0)(1,0,0)[12] intercept : AIC=2074.156, Time=0.61 sec
ARIMA(4,1,0)(2,0,1)[12] intercept : AIC=2054.138, Time=2.02 sec
ARIMA(4,1,0)(1,0,2)[12] intercept : AIC=2054.133, Time=1.71 sec
ARIMA(4,1,0)(0,0,0)[12] intercept : AIC=2434.998, Time=0.14 sec
ARIMA(4,1,0)(0,0,2)[12] intercept : AIC=2212.495, Time=0.64 sec
ARIMA(4,1,0)(2,0,0)[12] intercept : AIC=inf, Time=1.74 sec
ARIMA(4,1,0)(2,0,2)[12] intercept : AIC=2055.891, Time=2.54 sec
ARIMA(5,1,0)(1,0,1)[12] intercept : AIC=2054.146, Time=0.81 sec
ARIMA(4,1,1)(1,0,1)[12] intercept : AIC=2047.346, Time=0.98 sec

```

```

ARIMA(4,1,1)(0,0,1)[12] intercept : AIC=2244.995, Time=0.81 sec
ARIMA(4,1,1)(1,0,0)[12] intercept : AIC=2066.692, Time=0.62 sec
ARIMA(4,1,1)(2,0,1)[12] intercept : AIC=2049.240, Time=1.92 sec
ARIMA(4,1,1)(1,0,2)[12] intercept : AIC=2049.195, Time=1.67 sec
ARIMA(4,1,1)(0,0,0)[12] intercept : AIC=2429.816, Time=0.29 sec
ARIMA(4,1,1)(0,0,2)[12] intercept : AIC=2190.717, Time=1.36 sec
ARIMA(4,1,1)(2,0,0)[12] intercept : AIC=2052.093, Time=1.34 sec
ARIMA(4,1,1)(2,0,2)[12] intercept : AIC=2050.407, Time=2.90 sec
ARIMA(3,1,1)(1,0,1)[12] intercept : AIC=2046.703, Time=1.62 sec
ARIMA(3,1,1)(0,0,1)[12] intercept : AIC=2243.681, Time=0.56 sec
ARIMA(3,1,1)(1,0,0)[12] intercept : AIC=2069.207, Time=0.63 sec
ARIMA(3,1,1)(2,0,1)[12] intercept : AIC=2048.641, Time=2.03 sec
ARIMA(3,1,1)(1,0,2)[12] intercept : AIC=2048.612, Time=1.65 sec
ARIMA(3,1,1)(0,0,0)[12] intercept : AIC=2403.685, Time=0.19 sec
ARIMA(3,1,1)(0,0,2)[12] intercept : AIC=2192.044, Time=0.98 sec
ARIMA(3,1,1)(2,0,0)[12] intercept : AIC=2051.799, Time=1.43 sec
ARIMA(3,1,1)(2,0,2)[12] intercept : AIC=2050.702, Time=1.64 sec
ARIMA(2,1,1)(1,0,1)[12] intercept : AIC=2049.701, Time=0.63 sec
ARIMA(3,1,2)(1,0,1)[12] intercept : AIC=2048.030, Time=0.70 sec
ARIMA(2,1,2)(1,0,1)[12] intercept : AIC=2049.503, Time=0.66 sec
ARIMA(4,1,2)(1,0,1)[12] intercept : AIC=inf, Time=1.17 sec
ARIMA(3,1,1)(1,0,1)[12] : AIC=2044.718, Time=0.43 sec
ARIMA(3,1,1)(0,0,1)[12] : AIC=2241.761, Time=0.36 sec
ARIMA(3,1,1)(1,0,0)[12] : AIC=2067.214, Time=0.44 sec
ARIMA(3,1,1)(2,0,1)[12] : AIC=2046.656, Time=1.13 sec
ARIMA(3,1,1)(1,0,2)[12] : AIC=2046.626, Time=1.27 sec
ARIMA(3,1,1)(0,0,0)[12] : AIC=2401.751, Time=0.13 sec
ARIMA(3,1,1)(0,0,2)[12] : AIC=2190.081, Time=0.84 sec
ARIMA(3,1,1)(2,0,0)[12] : AIC=2049.809, Time=0.74 sec
ARIMA(3,1,1)(2,0,2)[12] : AIC=2048.601, Time=2.14 sec
ARIMA(2,1,1)(1,0,1)[12] : AIC=2047.721, Time=0.50 sec
ARIMA(3,1,0)(1,0,1)[12] : AIC=2053.191, Time=0.32 sec
ARIMA(4,1,1)(1,0,1)[12] : AIC=2045.363, Time=0.66 sec
ARIMA(3,1,2)(1,0,1)[12] : AIC=2046.045, Time=0.54 sec
ARIMA(2,1,0)(1,0,1)[12] : AIC=2072.145, Time=0.24 sec
ARIMA(2,1,2)(1,0,1)[12] : AIC=2047.518, Time=0.67 sec
ARIMA(4,1,0)(1,0,1)[12] : AIC=2050.152, Time=0.38 sec
ARIMA(4,1,2)(1,0,1)[12] : AIC=inf, Time=1.18 sec

```

Best model: ARIMA(3,1,1)(1,0,1)[12]

Total fit time: 91.633 seconds

O melhor modelo obtido, neste caso, foi SARIMA(3, 1, 1)(1, 0, 1)₁₂. Abaixo, apresentamos o dataframe gerado com as previsões e intervalo de confiança de 95%.

	Previsao_SARIMA	Limite Inferior (95%)	Limite Superior (95%)
Data			
2024-07-31	92.787790	69.049343	116.526237
2024-08-31	92.877040	62.934681	122.819399
2024-09-30	87.528520	54.544546	120.512494
2024-10-31	90.392059	56.455709	124.328409
2024-11-30	95.167153	60.668320	129.665986
2024-12-31	132.285444	97.193078	167.377811
2025-01-31	179.171214	143.257222	215.085207
2025-02-28	133.513755	96.598385	170.429125
2025-03-31	100.657220	62.688300	138.626141
2025-04-30	86.058453	47.084399	125.032507
2025-05-31	91.117560	51.211237	131.023882
2025-06-30	92.869768	52.086432	133.653104
2025-07-31	95.816966	49.402070	142.231862
2025-08-31	95.409225	45.260448	145.558002
2025-09-30	90.108091	37.326229	142.889953
2025-10-31	92.763709	38.383005	147.144413
2025-11-30	97.356969	41.654221	153.059717
2025-12-31	132.608499	75.587781	189.629217
2026-01-31	177.082905	118.624417	235.541392
2026-02-28	133.823569	73.847918	193.799220
2026-03-31	102.675481	41.176680	164.174283
2026-04-30	88.827035	25.852740	151.801330
2026-05-31	93.613530	29.223877	158.003182
2026-06-30	95.273181	29.517741	161.028622

Figura 24: Saída do Programa: Dataframe com as previsões para toda a série - SARIMA

Também reutilizamos alguns trechos de código do trabalho 01 e pudemos obter as previsões com intervalo de confiança de 95% para o melhor modelo ETS.

```

1 # Comparando com o modelo ETS(A,Ad,A)
2 best_ets_model = ETSModel(data['Vendas'], error='add', trend=
  'add',
3                               seasonal='add', damped_trend=True,
4                               seasonal_periods=12).fit()
5
6 # Previsão para 24 meses a frente com ETS
7 forecast_ets = best_ets_model.forecast(steps=forecast_horizon
8 )
9
10 # Intervalo de confiança para o ETS
11 sigma_ets = np.std(best_ets_model.resid)
12 lower_bound_ets = forecast_ets - 1.96 * sigma_ets
13 upper_bound_ets = forecast_ets + 1.96 * sigma_ets
14
15 # Criar um DataFrame com as previsões do ETS
16 forecast_ets_df = pd.DataFrame({
17     'Data': forecast_index,
18     'Previsao_ETS': forecast_ets,
19     'Limite Inferior (95%)': lower_bound_ets,

```

```

18     'Limite Superior (95%)': upper_bound_ets
19 }).set_index('Data')

```

	Previsao_ETS	Limite Inferior (95%)	Limite Superior (95%)
Data			
2024-07-31	92.308366	65.003894	119.612838
2024-08-31	92.147782	64.843310	119.452254
2024-09-30	86.665721	59.361249	113.970193
2024-10-31	89.676549	62.372077	116.981021
2024-11-30	93.033202	65.728730	120.337674
2024-12-31	133.383398	106.078926	160.687870
2025-01-31	186.287814	158.983342	213.592286
2025-02-28	132.490503	105.186031	159.794975
2025-03-31	96.577224	69.272753	123.881696
2025-04-30	83.019903	55.715431	110.324375
2025-05-31	87.389475	60.085003	114.693947
2025-06-30	89.437787	62.133315	116.742259
2025-07-31	92.208362	64.903890	119.512834
2025-08-31	92.049778	64.745307	119.354250
2025-09-30	86.569677	59.265205	113.874149
2025-10-31	89.582426	62.277954	116.886898
2025-11-30	92.940961	65.636489	120.245433
2025-12-31	133.293002	105.988530	160.597474
2026-01-31	186.199227	158.894755	213.503699
2026-02-28	132.403687	105.099215	159.708159
2026-03-31	96.492145	69.187673	123.796617
2026-04-30	82.936525	55.632053	110.240997
2026-05-31	87.307764	60.003292	114.612236
2026-06-30	89.357710	62.053238	116.662182

Figura 25: Saída do Programa: Dataframe com as previsões para toda a série - ETS

E, por fim, geramos um gráfico para enfatizar os resultados das previsões geradas e para fins de comparação e análise.

```

1 # Plotando as previsoes e comparando os intervalos de
  confianca
2 plt.figure(figsize=(10, 6))
3
4 # Plot dados reais
5 plt.plot(data.index, data['Vendas'], label="Dados Reais",
6         color='black')
7
8 # Plot SARIMA
9 plt.plot(forecast_sarima_df.index,
10         forecast_sarima_df['Previsao_SARIMA'], label="
11         Previsao SARIMA", color='blue')
12 plt.fill_between(forecast_sarima_df.index,
13                 forecast_sarima_df['Limite Inferior (95%)'],
14                 forecast_sarima_df['Limite Superior (95%)'],

```

```

13         color='blue', alpha=0.2, label="IC 95%
           SARIMA")
14
15 # Plot ETS
16 plt.plot(forecast_ets_df.index,
17          forecast_ets_df['Previsao_ETS'], label="Previsao ETS
           ", color='orange')
18 plt.fill_between(forecast_ets_df.index,
19                  forecast_ets_df['Limite Inferior (95%)'],
20                  forecast_ets_df['Limite Superior (95%)'],
21                  color='orange', alpha=0.2, label="IC 95% ETS
           ")
22
23 plt.title("Comparacao de Previsoes SARIMA e ETS com Intervalo
           de Confianca de 95%")
24 plt.legend()
25 plt.grid(True)
26 plt.show()

```

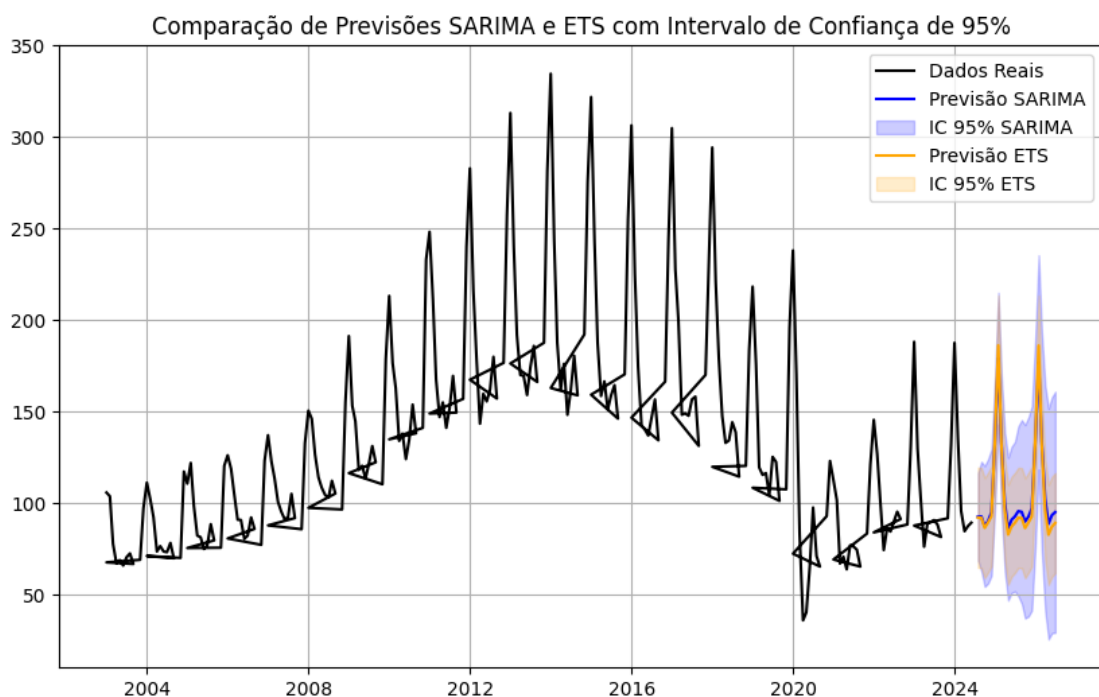


Figura 26: Gráfico comparativo das previsões - modelos ETS e SARIMA

Para interpretar os resultados obtidos com o melhor modelo SARIMA e compará-lo com o modelo $ETS(A,Ad,A)$, destacamos os seguintes pontos importantes:

1. Modelo SARIMA

O melhor modelo selecionado foi o $ARIMA(3,1,1)(1,0,1)[12]$, o que indica a presença de sazonalidade com um período de 12 meses. As previsões deste modelo mostram uma variação mais acentuada em alguns momentos, como pode ser observado tanto na tabela de previsões quanto no gráfico.

Os intervalos de confiança para o modelo SARIMA tendem a se expandir ao longo do tempo, refletindo uma maior incerteza à medida que o horizonte de previsão se amplia. No gráfico, o SARIMA parece seguir o padrão dos dados históricos, mas com uma maior faixa de incerteza quando comparado ao ETS.

2. Modelo ETS(A,Ad,A)

O modelo $ETS(A,Ad,A)$ foi o melhor selecionado a partir da lista anterior. Este modelo captura variações aditivas e a sazonalidade de forma amortecida (*damped*).

As previsões do modelo ETS mostram um intervalo de confiança um pouco mais estreito em relação ao SARIMA, indicando que o modelo ETS apresenta uma maior "confiança" em suas previsões. Isso sugere que o modelo ETS capturou a estrutura de sazonalidade e tendência da série de maneira mais eficiente.

No gráfico, a linha de previsão do ETS segue mais de perto a média dos valores reais, e os intervalos de confiança são significativamente mais contidos, sugerindo previsões mais estáveis.

3. Comparação

Ao comparar as previsões dos modelos, tanto pela tabela quanto pelo gráfico, podemos concluir que:

- O modelo **SARIMA** apresenta maior variabilidade em suas previsões a longo prazo, como indicado pelos amplos intervalos de confiança. Essa variabilidade pode ser atribuída à flexibilidade do modelo em capturar padrões complexos por meio de componentes autoregressivos e médias móveis.
- O modelo **ETS(A,Ad,A)**, por outro lado, gera previsões com intervalos de confiança mais estreitos, o que reflete uma maior robustez nas previsões de curto e médio prazo, possivelmente devido ao ajuste adequado da tendência amortecida e sazonalidade aditiva.

Então, o modelo $ETS(A,Ad,A)$ parece ser o mais adequado para essa série temporal, proporcionando previsões com menor incerteza. O SARIMA, apesar de sua flexibilidade, apresenta uma maior amplitude de incerteza nas previsões a longo prazo.

Dado que o foco está em obter previsões mais precisas, o **ETS(A,Ad,A)** deve ser considerado a escolha preferível, especialmente para séries com sazonalidade e tendências de médio prazo mais bem definidas.